

Heuristic approaches for the two- and three-dimensional knapsack packing problem

Jens Egeblad*, David Pisinger

Department of Computer Science, University of Copenhagen, Computer Science Universitetsparken 1, 2100 Copenhagen, Denmark

Available online 14 December 2007

Abstract

The maximum profit two- or three-dimensional knapsack packing problem packs a maximum profit subset of some given rectangles or boxes into a larger rectangle or box of fixed dimensions. Items must be orthogonally packed, but no other restriction is imposed to the problem. We present a new iterative heuristic for the two-dimensional knapsack problem based on the sequence pair representation proposed by Murata et al. [VLSI module packing based on rectangle-packing by the sequence pair. IEEE Transaction on Computer Aided Design of Integrated Circuits and Systems 1996;15:1518–24] using a semi-normalized packing algorithm by Pisinger [Denser packings obtained in $O(n \log \log n)$ time. INFORMS Journal on Computing 2007;19:395–405]. Solutions are represented as a pair of sequences. In each iteration, the sequence pair is modified and transformed to a packing in order to evaluate the objective value. Simulated annealing is used to control the heuristic. A novel abstract representation of box placements, called sequence triple, is used with a similar technique for the three-dimensional knapsack problem. The heuristic is able to handle problem instances where rotation is allowed. Comprehensive computational experiments which compare the developed heuristics with previous approaches indicate very promising results for both two- and three-dimensional problems.

© 2007 Elsevier Ltd. All rights reserved.

Keywords: Cutting and packing; Two-dimensional knapsack problems; Three-dimensional knapsack problem; Heuristics; Simulated annealing; Sequence pair; Sequence-triple

1. Introduction

Assume that we are given a set of n rectangles $j = 1, \dots, n$, each having a width w_j , height h_j and profit p_j and a rectangular plate having width W and height H . The *maximum profit two-dimensional knapsack packing* (2DKP) problem assigns a subset of the rectangles onto the plate such that the associated profit sum is maximized. All coefficients are assumed to be nonnegative integers, and the rectangles may not be rotated. A packing of rectangles on the plate is feasible if no two rectangles overlap, and if no part of any rectangle exceeds the plate.

The *maximum profit three-dimensional knapsack packing* (3DKP) problem assigns a subset of boxes each with dimensions w_j, h_j, d_j into a larger box with dimensions W, H and D .

The problem has direct applications in various packing and cutting problems where the task is to use the space or material in an optimal way. The 2DKP problem also appears as pricing problem when solving the two-dimensional bin-packing problem [1–3]. 2DKP and 3DKP are NP-hard in the strong sense, which can be shown by reduction from

* Corresponding author. Tel.: +45 35 32 14 00; fax: +45 35 32 14 01.

E-mail addresses: jegeblad@diku.dk (J. Egeblad), pisinger@diku.dk (D. Pisinger).

the one-dimensional bin-packing problem. An extensive survey on cutting and packing as well as a useful classification of these problems was developed by Wascher et al. [4].

The problems we consider in this paper can be classified as two- and three-dimensional rectangular single knapsack problems (SKP) according to the typology of Wascher et al. [4]. The items considered are strongly heterogeneous and we consider problems with and without rotation.

A related problem is the constrained two-dimensional orthogonal non-guillotine cutting problem. Here equal items are grouped in types and for each item-type there are both a lower bound and an upper bound on the number of that type required in the solution. Therefore the constrained non-guillotine cutting problem may be seen as a generalization of the orthogonal knapsack packing problem. Instances for the constrained non-guillotine cutting problem are often weakly heterogeneous, and solution methods commonly take advantage here of.

Integer Programming formulations of the 2DKP have been presented by Beasley [5], Hadjiconstantinou and Christofides [6] and Boschetti et al. [7] among others.

Fekete and Schepers [1,8,9] solved the 2- and 3DKP through a branch-and-bound algorithm which assigns items to the knapsack without specifying the position of the rectangles. For each assignment of items a two-dimensional packing problem is solved, deciding whether a feasible assignment of coordinates to the items is possible such that they all fit into the knapsack without overlaps. An advanced graph representation was used for solving the latter problem. Baldacci and Boschetti [10] used a similar approach but introduced new reduction tests and a cutting-plane approach to compute more effective bounds. Pisinger and Sigurd [3] solved the 2DKP through a branch-and-cut approach in which an ordinary one-dimensional knapsack problem is used to select the most profitable items whose overall area does not exceed the area of the plate. Having selected the most profitable items, a two-dimensional packing problem in decision form is solved, through constraint programming. If all items can be placed in the knapsack the algorithm terminates, otherwise an inequality is added to the one-dimensional knapsack stating that not all the current items can be selected simultaneously, and the process is repeated. Finally, Caprara and Monaci [11] developed a branch-and-bound algorithm for the 2DKP. The algorithm is based on a branch-and-bound scheme which assigns items to the knapsack without specifying the position of each item, followed by a feasibility check. The latter is done using an enumeration scheme from Martello et al. [12].

Several authors have also applied heuristics to the constrained non-guillotine packing variant of the problem. Lai and Chan [13,14] use simulated annealing and genetic algorithms. Solutions are represented as a sequence of items that are transformed into a placement. Only limited computational results are reported. The work by Leung et al. [15,16] is based on that of Lai and Chan and the bottom-left placement procedure introduced by Jakobs [17]. Beasley [18] described a heuristic based on genetic algorithms capable of efficiently generating good solutions for instances with up to 4000 pieces. However, the heuristic is unable to reproduce known optimal solutions for smaller instances. Intermediate solutions explicitly state coordinates of rectangles, and overlap of items are allowed during the solution process. A penalty in the objective function ensures that overlap is minimized. More recently, Alvarez-Valdes et al. applied both of the meta-heuristics GRASP and Tabu-search [19,20] to the problem and were able to achieve very impressive results on the data used by Beasley.

In the present paper we first present an IP formulation of the 2- and 3DKP. In Section 3 we describe the sequence pair representation, which we use in Section 4 with a simple local search neighborhood controlled by simulated annealing to solve 2DKP. In Section 5 we introduce a novel abstract representation of box placements in three dimensions and use the same methods as for two dimensions to solve 3DKP. Finally, in Section 6, we present our results on existing and new benchmark instances for 2- and 3DKP.

2. Integer programming formulation of the problem

In the following we show an integer programming formulation of the 3DKP. A formulation of 2DKP easily follows by removing variables and constraints for the third dimension.

We will introduce the decision variable s_i to indicate whether box i is packed within the knapsack box. The coordinates of box i are (x_i, y_i, z_i) , meaning that the lower left back corner of the box is located at this position. If a rectangle is not packed within the knapsack we may assume that $(x_i, y_i, z_i) = (0, 0, 0)$. As no part of a packed box may exceed the knapsack, we have the obvious constraints

$$0 \leq x_i \leq W - w_i, \quad 0 \leq y_i \leq H - h_i, \quad 0 \leq z_i \leq D - d_i. \quad (1)$$

We introduce the binary decision variables ℓ_{ij} (left), r_{ij} (right), u_{ij} (under), o_{ij} (over), b_{ij} (behind) and f_{ij} (in-front), to indicate the relative position of boxes i, j where $i < j$. To ensure that no two packed boxes i, j overlap we will demand that

$$\ell_{ij} + r_{ij} + u_{ij} + o_{ij} + b_{ij} + f_{ij} = 1, \quad (2)$$

whenever $s_i = s_j = 1$. Depending on the relative position of two rectangles the coordinates must satisfy the following inequalities:

$$\begin{aligned} \ell_{ij} = 1 &\Rightarrow x_i + w_i \leq x_j, & r_{ij} = 1 &\Rightarrow x_j + w_j \leq x_i, \\ u_{ij} = 1 &\Rightarrow y_i + h_i \leq y_j, & o_{ij} = 1 &\Rightarrow y_j + h_j \leq y_i, \\ b_{ij} = 1 &\Rightarrow z_i + d_i \leq z_j, & f_{ij} = 1 &\Rightarrow z_j + d_j \leq z_i. \end{aligned} \quad (3)$$

The problem may now be formulated as

$$\begin{aligned} \max \quad & \sum_{i=1}^n p_i s_i \\ \text{s.t.} \quad & \ell_{ij} + r_{ij} + u_{ij} + o_{ij} + b_{ij} + f_{ij} = s_i + s_j - 1, \quad i, j = 1, \dots, n, \\ & x_i - x_j + W\ell_{ij} \leq W - w_i, \quad i, j = 1, \dots, n, \\ & x_j - x_i + Wr_{ij} \leq W - w_j, \quad i, j = 1, \dots, n, \\ & y_i - y_j + Hu_{ij} \leq H - h_i, \quad i, j = 1, \dots, n, \\ & y_j - y_i + Ho_{ij} \leq H - h_j, \quad i, j = 1, \dots, n, \\ & z_i - z_j + Db_{ij} \leq D - d_i, \quad i, j = 1, \dots, n, \\ & z_j - z_i + Df_{ij} \leq D - d_j, \quad i, j = 1, \dots, n, \\ & 0 \leq x_i \leq W - w_i, \quad i = 1, \dots, n, \\ & 0 \leq y_i \leq H - h_i, \quad i = 1, \dots, n, \\ & 0 \leq z_i \leq D - d_i, \quad i = 1, \dots, n, \\ & \ell_{ij}, r_{ij}, u_{ij}, o_{ij}, b_{ij}, f_{ij} \in \{0, 1\}, \quad i = 1, \dots, n, \\ & s_i \in \{0, 1\}, \quad i = 1, \dots, n, \\ & x_i, y_i, z_i \geq 0, \quad i = 1, \dots, n. \end{aligned} \quad (4)$$

The first constraint ensures that if boxes i and j are packed, then they must be located left, right, under, over, behind or in-front of each other as stated in (2). The next six constraints are just linear versions of the constraints (3). The last three inequalities correspond to the constraints (1).

The MIP-model has $6n^2 + n$ binary decision variables and $3n$ continuous variables. Although the size of $O(n^2)$ binary variables is not alarming, the problem is difficult to solve. This is mainly due to the use of conditional constraints (3), as these will loose their effect when solving the LP-relaxation, and thus bounds from LP-relaxation are in general far from the MIP-optimal solution value.

3. Sequence pairs

Murata et al. [21] presented an abstract representation of two-dimensional rectangle packings based on sequence pairs. The problem they consider is the minimum area enclosing rectangle packing problem. In the abstract representation every compact packing can be represented by two permutations of the numbers $\{1, 2, \dots, n\}$ where each number represents a rectangle in the problem instance. The pair of permutations is called a *sequence pair* (A, B) .

For a given packing, the two permutations A and B are found as follows: We use the relation $\mathcal{A}_{ij}$ to denote that item i precedes j in sequence A . We define

$$(x_i + w_i \leq x_j \vee y_i \geq y_j + h_j) \Leftrightarrow \mathcal{A}_{ij}. \quad (5)$$

In a similar way we use the relation $\mathcal{B}_{ij}$ to denote that item i precedes j in sequence B , defining

$$(x_i + w_i \leq x_j \vee y_i + h_i \leq y_j) \Leftrightarrow \mathcal{B}_{ij}. \quad (6)$$

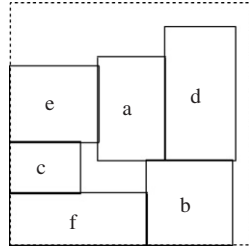


Fig. 1. A packing represented by sequence $A = \langle e, c, a, d, f, b \rangle$ and sequence $B = \langle f, c, b, e, a, d \rangle$.

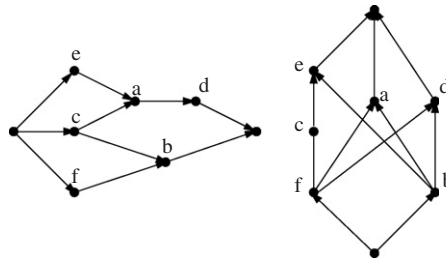


Fig. 2. Constraint graphs corresponding to the sequence pair $(A, B) = (\langle e, c, a, d, f, b \rangle, \langle f, c, b, e, a, d \rangle)$.

Each of the two relations $\mathcal{A}, \mathcal{B}$ given by (5) and (6) defines a semi-ordering, and hence for a given packing the two permutations A and B can easily be found by repeatedly choosing one (of possibly more) minimum elements. Fig. 1 illustrates a packing and a corresponding sequence pair (A, B) .

From (5) and (6) we immediately see that if item i precedes item j in both sequences, then i must be placed left of j . If i succeeds j in sequence A but i precedes j in sequence B then i must be placed under j . Formally we have

$$\mathcal{A}_{ij} \wedge \mathcal{B}_{ij} \Rightarrow i \text{ is left of } j, \quad (7)$$

$$\neg \mathcal{A}_{ij} \wedge \mathcal{B}_{ij} \Rightarrow i \text{ is under } j, \quad (8)$$

where we use the terminology $\neg \mathcal{A}_{ij}$ to denote $\mathcal{A}_{ji}$.

The implications (7) and (8) can be used to derive a pair of constraint graphs as illustrated in Fig. 2. In both graphs the nodes correspond to the items and edges indicate which rectangles should be placed left of each other (respectively under each other). In the first graph we have an edge from i to j if and only if item i should be placed left of j ($\mathcal{A}_{ij} \wedge \mathcal{B}_{ij}$). In the second graph we have an edge from i to j if and only if item i should be placed under j ($\neg \mathcal{A}_{ij} \wedge \mathcal{B}_{ij}$). Redundant edges are removed from the figure for clarity. Traversing the nodes in topological order while assigning coordinates to the items, a packing (i.e. the coordinates of the items) can be obtained in $O(n^2)$ time. Tang et al. [22,23] showed how the same packing can be derived without explicitly defining the constraint graph, but by finding weighted longest common subsequences in the sequence pair.

Pisinger [24] further improved the algorithm, by presenting an algorithm which transforms a sequence pair to a *semi-normalized* packing in time $O(n \log \log n)$. A *normalized* packing is a packing where the items are packed according to the sequence B and where each new item is placed such that it touches an already placed item on its left side, and an already placed item on its lower side. A *semi-normalized* packing is a packing where the items are packed according to the sequence B and where each new item is placed such that it touches the *contour* of the already placed items both from left and from below. The difference between a packing based on the ordinary transformation and the semi-normalized packing of the transformation by Pisinger is illustrated in Fig. 3.

4. Sequence pairs for two-dimensional knapsack packing

Let any sequence pair represent a feasible solution to the 2DKP. To evaluate the solution we transform the sequence pair into a packing. The solution value is sum of the profit values of those items which are located completely within the

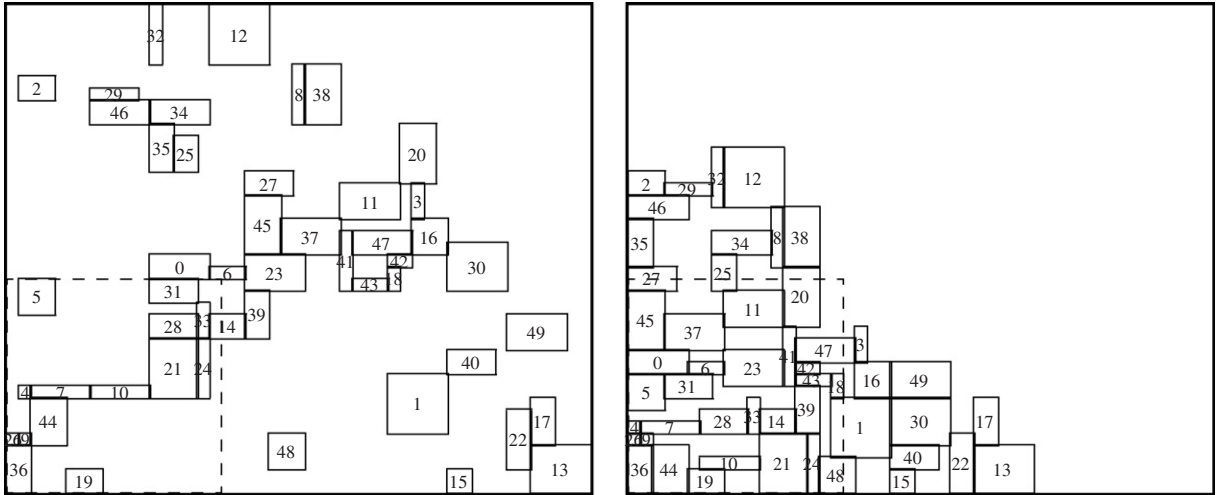


Fig. 3. Transformation of a sequence pair to a packing using the ordinary transformation (left) and using the semi-normalized transformation (right).

knapsack $W \times H$ of the transformed packing. Fig. 3 illustrates two such packings which arose from the conventional and semi-normalized transformation of the sequence pair. The solution values of each packing is the sum of the profits of items within the dashed lines.

The transformation from sequence pair to packing may be stopped as soon as the contour of already placed items is completely outside the knapsack. For problems where only a small fraction of items fit inside the knapsack, this can save substantial time, since generating the packing will take an amount of time which is roughly equal to the time required to place only the subset of items which are inside the knapsack.

This section is organized as follows: In Section 4.1 we describe our heuristic in more detail. In Section 4.2 we describe how to accommodate problems where rotation is allowed and in Section 4.3 we describe how problem instances can be simplified during the solution process.

4.1. Simulated annealing

To solve the 2DKP, we use the meta-heuristic simulated annealing which works well in cooperation with the sequence pair representation [21–24].

In this setting we repeatedly make a small modification to the sequence pair, transform the sequence pair into a packing, evaluate the profit of the corresponding packing and accept the solution depending on the outcome. In simulated annealing any non-improving solution is accepted with probability that decreases over time. An outline of the algorithm is found in Fig. 4.

Our variant of simulated annealing is as follows: At any given time the temperature is evaluated as $1/(t_0 + t_s \cdot a)$ where t_0 is a start time-value, t_s is a time-step value and a is the number of accepted solutions. The temperature depends on the time, so the higher the $t_0 + t_s \cdot a$ is, the lower is the current temperature. The temperature is decreased only when a new solution is accepted.

The neighborhood $N(s)$ of a solution $s = (A, B)$ is defined as one of the following three permutations: Either exchange two items in sequence A ; exchange two items in sequence B ; or exchange two items in both sequences A and B . The items are selected randomly.

4.2. Rotations

Few papers consider exact algorithms for packing problems where rotation is allowed. A possible explanation could be the increased size of the solution space and the lack of high-quality upper bounds. In our heuristic, rotations are easy to handle as we may represent each packing by the triple (A, B, R) . Here (A, B) is the sequence pair and $R = (r_1, \dots, r_n)$ is a binary vector of length n . In a placement of (A, B, R) item i is rotated $r_i \cdot 90^\circ$. If rotation is

```

choose initial incumbent solution  $s \in S$ 
choose initial time  $t_0$ 
choose time step  $t_s$ 
 $a := 0$ 
repeat
  choose  $s' \in N(s)$ 
  if  $f(s') \leq f(s)$  then
     $accept := true$ 
  else
     $p := \text{rand}(0,1)$ 
     $T := \frac{1}{t_0 + t_s \cdot a}$ 
     $\Delta := \frac{f(s) - f(s')}{f(s)}$ 
    if  $p < e^{-\frac{\Delta}{T}}$  then
       $accept := true$ 
    end
  end
  if  $accept$  then
     $s := s'$ 
     $a := a + 1$ 
  end
until stopping-criteria
return  $s$ 

```

Fig. 4. Simulated annealing heuristic.

allowed the neighborhood $N(s)$ of our heuristic is extended with a fourth permutation: change the rotation flag of an item in R .

4.3. Removing items

In instances where only a small fraction of the items can fit inside the knapsack, it can be advantageous to a priori remove some items which provably will not be selected in an optimal solution. Our approach is based on standard techniques for reducing the number of items in a 0–1 knapsack problem [25] and is similar to the one presented in [20].

Let the one-dimensional relaxation of the (2DKP) be given by the following 0–1 knapsack problem (1DKP):

$$\begin{aligned}
 \max \quad & \sum_{j=1}^n p_j x_j \\
 \text{s.t.} \quad & \sum_{j=1}^n (w_j h_j) x_j \leq WH \\
 & x_j \in \{0, 1\}, \quad j = 1, \dots, n.
 \end{aligned} \tag{9}$$

Assume that z^* is the currently best known solution to (2DKP), then clearly z^* is a lower bound for (1DKP), and an optimal solution to (1DKP) is also an upper bound on z^* .

Now, assume that we have an upper bound u_j^1 for (1DKP) with the additional constraint that $x_j = 1$. If $u_j^1 \leq z^*$ then we know that item j will not be chosen in an improved solution of (1DKP), and hence neither in an improved solution of (2DKP).

As our upper bound we have chosen to use the Dembo and Hammer [26] upper bound u_{DH} which can be calculated as follows: Sort the items in (1DKP) according to nonincreasing efficiency $p_j/(w_j h_j)$ and fill the knapsack in a greedy way until the first item s (split item) does not fit into the knapsack. Then the Dembo and Hammer upper bound is given by $u_{DH} = \sum_{i=1}^{s-1} p_i + (WH - \sum_{i=1}^{s-1} w_i h_i) p_s / (w_s h_s)$. If we fix a variable $x_j = 1$ the upper bound becomes $u_j^1 = u_{DH} + p_j - (w_j h_j) p_s / (w_s h_s)$ which can be calculated in constant time.

During the simulated annealing, we run this test whenever we encounter an improving solution z^* and remove every item j for which $u_j^1 \leq z^*$ from the problem.

5. Three dimensions

For the three-dimensional problem we will consider a new representation which like the sequence pair for two dimensions will contain the relative box placements for three dimensions. We call the representation sequence triple since it consists of three sequences. Not all three-dimensional packings are obtainable with this representation but we will prove that a large subset of all normalized packings, known as fully robot packable packings, may be represented.

A *robot packing* is a packing which can be achieved by successively placing boxes starting from the bottom-left-behind corner, and such that each box is in-front of, right of, or over each of the previously placed boxes [27]. Robot packings are motivated by several industrial applications, where boxes have to be packed by robots equipped with a rectangular “hand” parallel to the base of the large box. To avoid collisions between the hand and the boxes, it is demanded that no already packed box blocks for the “hand” movement. In [27] it is shown that the quality of a packing is seldom affected by restricting the solution space to the set of robot packings.

A packing is a *fully robot packable packing* if all six 90° rotations of it are robot packings or, equivalently, the robot criteria is satisfied no matter which corner is selected as start corner instead of the bottom-left-behind corner.

This section is organized as follows: First we describe the sequence triple representation in detail in Section 5.1. Then we describe an algorithm to transform a sequence triple to a packing in Section 5.2. Finally, in Section 5.3, we describe how the same simulated annealing strategy we use for the sequence pair is applied to the sequence triple to form a heuristic for 3DKP.

5.1. Sequence triple

A given fully robot packable packing is represented by three sequences A , B and C where each sequence is a permutation of the n boxes. For any sequence X we define the relation $\mathcal{X}_{ij}$ to mean that i is before j in sequence X . For convenience we use the notation $\neg \mathcal{X}_{ij} \Leftrightarrow \mathcal{X}_{ji}$.

In a similar way as in Section 3 we define the relation $\mathcal{A}_{ij}$ by

$$(x_i + w_i \leq x_j \vee y_i \geq y_j + h_j \vee z_i \geq z_j + d_j) \Leftrightarrow \mathcal{A}_{ij}. \quad (10)$$

In other words $\mathcal{A}_{ij}$ iff i is located left, over or in-front of j . Using the formulation (2) we have $\mathcal{A}_{ij} \Leftrightarrow \ell_{ij} + o_{ij} + f_{ij} = 1$.

Relation $\mathcal{B}_{ij}$ is defined by

$$(x_i \leq x_j + w_j \vee y_i \leq y_j + h_j \vee z_i \leq z_j + d_j) \Leftrightarrow \mathcal{B}_{ij}. \quad (11)$$

This means $\mathcal{B}_{ij}$ iff i is located left, under or behind of j . The relation can be expressed as $\mathcal{B}_{ij} \Leftrightarrow \ell_{ij} + u_{ij} + b_{ij} = 1$.

Finally, relation $\mathcal{C}_{ij}$ is defined by

$$(x_i \geq x_j + w_j \vee y_i + h_i \leq y_j \vee z_i \geq z_j + d_j) \Leftrightarrow \mathcal{C}_{ij}. \quad (12)$$

In words, $\mathcal{C}_{ij}$ iff i is located right, under or in-front of j , which can be expressed as $\mathcal{C}_{ij} \Leftrightarrow r_{ij} + u_{ij} + f_{ij} = 1$.

Due to the definition of fully robot packable packings, there will always be an item which is located furthest left-over-behind. By removing this item and repeating the operation, we get the ordering of sequence A . In a similar way the orderings of B and C can be determined, as illustrated in Fig. 5 (the letters A , B , C on the figure indicate the directions which are used for defining the corresponding sequence of boxes). This shows that every fully robot packable packing can be represented by a sequence triple.

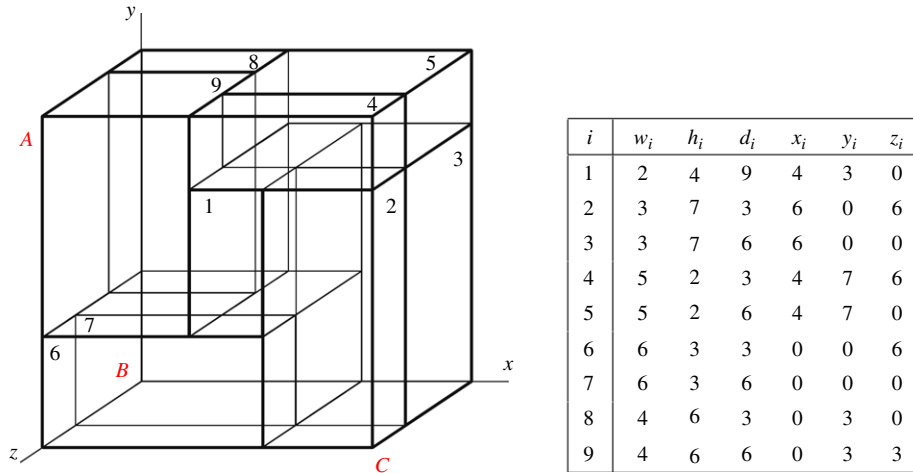


Fig. 5. A packing and the corresponding sequence triple $(A, B, C) = \{ \langle 9, 4, 8, 5, 1, 6, 2, 7, 3 \rangle, \langle 7, 8, 6, 9, 1, 3, 5, 2, 4 \rangle, \langle 2, 3, 6, 7, 1, 4, 5, 9, 8 \rangle \}$.

Using the relations

$$\mathcal{A}_{ij} \Leftrightarrow \ell_{ij} + o_{ij} + f_{ij} = 1,$$

$$\mathcal{B}_{ij} \Leftrightarrow \ell_{ij} + u_{ij} + b_{ij} = 1,$$

$$\mathcal{C}_{ij} \Leftrightarrow r_{ij} + u_{ij} + f_{ij} = 1,$$

(13)

we find that

$$\mathcal{A}_{ij} \wedge \neg \mathcal{B}_{ij} \wedge \mathcal{C}_{ij} \Leftrightarrow f_{ij} = 1,$$

$$\mathcal{A}_{ij} \wedge \mathcal{B}_{ij} \wedge \mathcal{C}_{ij} \Rightarrow \text{Infeasible},$$

$$\neg \mathcal{A}_{ij} \wedge \neg \mathcal{B}_{ij} \wedge \mathcal{C}_{ij} \Leftrightarrow r_{ij} = 1,$$

$$\neg \mathcal{A}_{ij} \wedge \mathcal{B}_{ij} \wedge \mathcal{C}_{ij} \Leftrightarrow u_{ij} = 1,$$

$$\mathcal{A}_{ij} \wedge \neg \mathcal{B}_{ij} \wedge \neg \mathcal{C}_{ij} \Leftrightarrow o_{ij} = 1,$$

$$\mathcal{A}_{ij} \wedge \mathcal{B}_{ij} \wedge \neg \mathcal{C}_{ij} \Leftrightarrow \ell_{ij} = 1,$$

$$\neg \mathcal{A}_{ij} \wedge \neg \mathcal{B}_{ij} \wedge \neg \mathcal{C}_{ij} \Rightarrow \text{Infeasible},$$

$$\neg \mathcal{A}_{ij} \wedge \mathcal{B}_{ij} \wedge \neg \mathcal{C}_{ij} \Leftrightarrow b_{ij} = 1.$$

(14)

Notice that $\mathcal{A}_{ij} \wedge \mathcal{B}_{ij} \wedge \mathcal{C}_{ij}$ and $\neg \mathcal{A}_{ij} \wedge \neg \mathcal{B}_{ij} \wedge \neg \mathcal{C}_{ij}$ cannot occur for any packing since we require $\ell_{ij} + u_{ij} + f_{ij} + r_{ij} + u_{ij} + b_{ij} = 1$. We have, however, chosen to assign these cases a meaning, such that every sequence triple has a corresponding packing. This leads to the following four implications, similar to (7) and (8), which are used to determine the relative box positions:

$$\mathcal{A}_{ij} \wedge \mathcal{B}_{ij} \wedge \neg \mathcal{C}_{ij} \Rightarrow i \text{ is left of } j, \quad (15)$$

$$\neg \mathcal{A}_{ij} \wedge \mathcal{B}_{ij} \wedge \mathcal{C}_{ij} \Rightarrow i \text{ is under } j, \quad (16)$$

$$\neg \mathcal{A}_{ij} \wedge \mathcal{B}_{ij} \wedge \neg \mathcal{C}_{ij} \Rightarrow i \text{ is behind } j, \quad (17)$$

$$\mathcal{A}_{ij} \wedge \mathcal{B}_{ij} \wedge \mathcal{C}_{ij} \Rightarrow i \text{ is behind } j. \quad (18)$$

Notice that both (17) and (18) impose that i must be behind j in the packing. The unfortunate consequence of this is that the representation is biased toward orderings in that direction which could have a negative impact on the solution process, but as we wish to let every sequence triple represent a packing, an arbitrary choice had to be done.

5.2. A placement algorithm

To find a placement (i.e. the coordinates of the boxes) corresponding to a sequence triple, we can construct three constraint graphs similar to Fig. 2: In the first graph we have an edge from item i to item j if i is located left of j (i.e. $\mathcal{A}_{ij} \wedge \mathcal{B}_{ij} \wedge \neg \mathcal{C}_{ij}$). In the second graph we have an edge from item i to item j if i is located under j (i.e. $\neg \mathcal{A}_{ij} \wedge \mathcal{B}_{ij} \wedge \mathcal{C}_{ij}$). In the last graph we have an edge from item i to item j if i is located behind j (i.e. $\neg \mathcal{A}_{ij} \wedge \mathcal{B}_{ij} \wedge \neg \mathcal{C}_{ij}$ or $\mathcal{A}_{ij} \wedge \mathcal{B}_{ij} \wedge \mathcal{C}_{ij}$). Traversing the nodes in topological order for each graph while assigning coordinates to the items, we find the location of all boxes in time $O(n^2)$.

By observing that $\mathcal{B}_{ij}$ is a necessary criteria for node i to precede node j in each of the three constraint graphs, we may actually omit the topological ordering as it is in each case given by the order of sequence B .

The first box in B is placed at $(x, y, z) = (0, 0, 0)$ and succeeding boxes are placed one by one according to the order of sequence B . At any time let P consist of all previously placed boxes. Now assume we wish to place box i . To determine the position of i we compare i with every box $j \in P$. Let $P_x \subseteq P$ be the subset of boxes which satisfy (15), (i.e. $\mathcal{A}_{ij} \wedge \mathcal{B}_{ij} \wedge \neg \mathcal{C}_{ij}$) let $P_y \subseteq P$ be the subset which satisfy (16), (i.e. $\neg \mathcal{A}_{ij} \wedge \mathcal{B}_{ij} \wedge \mathcal{C}_{ij}$) and let $P_z \subseteq P$ be the subset which satisfy (17) or (18) (i.e. $\neg \mathcal{A}_{ij} \wedge \mathcal{B}_{ij} \wedge \neg \mathcal{C}_{ij}$ or $\mathcal{A}_{ij} \wedge \mathcal{B}_{ij} \wedge \mathcal{C}_{ij}$). Now assign to i the coordinates (x_i, y_i, z_i) determined by

$$x_i = \max \left(0, \max_{j \in P_x} (x_j + w_j) \right), \quad (19)$$

$$y_i = \max \left(0, \max_{j \in P_y} (y_j + h_j) \right), \quad (20)$$

$$z_i = \max \left(0, \max_{j \in P_z} (z_j + d_j) \right). \quad (21)$$

Once a box has been placed it is inserted into P .

If we maintain a table in which the position of each box i in the three sequences A, B, C is saved, we can test whether $\mathcal{A}_{ij}, \mathcal{B}_{ij}$ or $\mathcal{C}_{ij}$ holds in constant time for two given boxes i, j . Since placing a box only requires comparison with every previously placed box, calculating (19) to (21) for a given box i can be done in $O(|P|) = O(n)$ time. Placing all n boxes then requires $O(n^2)$ time.

To speed up the placement procedure slightly we remove a box from P if it is completely “shaded” by a newly inserted box. A box j is shaded by a box i if $x_j + w_j \leq x_i + w_i$, $y_j + h_j \leq y_i + h_i$ and $z_j + d_j < z_i + d_i$, hence it does not affect the placement of future boxes, and by removing it we can avoid subsequent redundant checks.

5.3. Simulated annealing

To solve 3DKP, we use the same simulated annealing scheme used for two dimensions but with the three-dimensional sequence representation. The neighborhood is increased to accommodate the extra sequence and consists of the following permutations: (1) exchange two boxes from one of the sequences, (2) exchange two boxes in sequences A and B , (3) exchange two boxes in sequences A and C , (4) exchange two boxes in sequences B and C , (5) exchange two boxes in all sequences.

6. Computational experiments

The heuristic described in the previous sections was implemented in C++ using a modified version of the sequence pair algorithm by Pisinger [24] for two dimensions and an implementation of the placement algorithm for sequence triple described in Section 5.2 for three dimensions. The implementation was tested on a computer with an AMD Athlon 64 3800+ (2.4 GHz) processor with 2 GB of RAM using the GNU-C++ compiler (gcc 4.0). This section is divided into three parts: Section 6.1 considers tighter upper bounds to evaluate the quality of solutions; Section 6.2 deals with the 2DKP; Section 6.3 considers the 3DKP.

6.1. Bounds

To determine the quality of the solutions we compare solution values with upper bound introduced by Fekete et al. [28,9] which is based on *conservative scales*.

Table 1
The five different classes of the new ep2 instances

Class	Description	Width	Height
T	Tall. Rectangles are tall	$\left[1, \frac{1}{3} \cdot 100\right]$	$\left[\frac{2}{3} \cdot 100, 100\right]$
W	Wide. Rectangles are wide	$\left[\frac{2}{3} \cdot 100, 100\right]$	$\left[1, \frac{1}{3} \cdot 100\right]$
S	Square. Rectangles are square	$[1, 100]$	Equal to width
U	Uniform. Largest dimension is no more than 150% of the smallest	$\left[\frac{2}{3} \cdot 100, 100\right]$	$\left[\frac{2}{3} \cdot 100, 100\right]$
D	Diverse. Largest dimension can be up-to 100 times the smallest	$[1, 100]$	$[1, 100]$

Given an instance I of (3DKP) with container dimensions $W \times H \times D$ a new instance I' with dimensions $1 \times 1 \times 1$ is generated by scaling the dimensions of each item $i \in I$ by $\frac{1}{W}$, $\frac{1}{H}$ and $\frac{1}{D}$, respectively. Define the conservative scale:

$$u^{(k)}(x) = \begin{cases} x & \text{for } (k+1)x \in \mathbb{Z}, \\ \lfloor (k+1)x \rfloor \frac{1}{k} & \text{otherwise,} \end{cases}$$

and let $u^{(0)}(x) = x$. Now for each $j, k, l = 0, 1, \dots, 4$, instances are generated from I' by setting item i 's dimensions to $u^{(j)}(w'_i) \times u^{(k)}(h'_i) \times u^{(l)}(d'_i)$. The minimal optimal value of the one-dimensional relaxation of any of these instances is used as an upper bound of I . For an instance of (2DKP) similar bounds are determined by disregarding the third dimension.

For instances where rotation is allowed the upper bound from conservative scales is not valid, hence we use the weaker upper bound given by the optimal value of (1DKP) defined in (9).

6.2. 2D computational experiments

To test the 2DKP heuristic we used both classical instances from the literature and a new set of instances (described in Table 1). The instances were used for parameter tuning of the heuristic. Results are reported for instances both without and with rotation allowed.

6.2.1. Classical instances for 2DKP

We use the benchmarks instances considered by Fekete et al. [9], Caprara and Monaci [11] and Alvarez-Valdes et al. [19,20]. The instances are listed in Table 2.

The instances *beasley1–12* originate from [29]. The *cgcut* and *gcut* instances are guillotine-cut instances from [30] and [5], respectively. The guillotine-cut instance *wang20* is from [31]. The instances *3* to *CHL5* are also guillotine-cut instances by Hifi [32]. The data for *hadchr3* and *hadchr11* were presented in [6]. The instances *okp1–5* are by Fekete and Schepers [8].

To transform the *gcut* instances, exactly one rectangle was created in the 2DKP instance for each rectangle in the original instance. For the constrained instances b_i duplicates of each rectangle were created, where b_i is the maximum number of times rectangle i may be cut from the material.

In addition to the above instances, Beasley [18] presented a set of 630 instances (*ngcutfs*), which are listed in Table 3. In these, the number of distinct items, M , ranges between 40 and 1000, but items are duplicated Q times, with $Q \in \{1, 3, 4\}$. Therefore the total number of items, n , ranges from 40 to 4000.

6.2.2. New instances for 2DKP

To test the performance of the heuristic for problems where many rectangles can exist simultaneously in the knapsack, we have created 80 new instances. The rectangle dimensions in each instance belongs to one of five different classes which are listed in Table 1. Dimensions of the items are selected randomly from a uniform distribution between the first and last values of the intervals in the 'Width' and 'Height' columns of the table. The five classes are *tall* (T), *wide* (W), *square* (S), *uniform* (U) and *diverse* (D). The number of rectangles, n , in each instance is selected from the set $\{30, 50, 100, 200\}$. The rectangles may be *clustered* (C) and *random* (R). *Clustered* instances consists of only 20 rectangles which are duplicated appropriately, while in the *random* instances all rectangles are independently generated. Finally, the area of the bin is either 25% or 75% of the total area of the rectangles and the height of the

Table 2
Results for the classical benchmark instances

Instance	n	n_0	n_1	Optimal	Egeblad and Pisinger					Exact Methods Time		Time
					Best	Avg.	Worst	Best Time	Seed Time	Fek-Sch	Cap-Mon	
beasley1	10	5.3	5	164	164	164	164	≤ 0.02	30	≤ 0.02	–	0
beasley2	17	6.3	8	230	230	230	230	≤ 0.02	60	≤ 0.02	–	0
beasley3	21	7.6	6	247	247	247	247	≤ 0.02	60	≤ 0.02	–	0
beasley4	7	6.5	6	268	268	268	268	≤ 0.02	30	≤ 0.02	–	0
beasley5	14	6	7	358	358	358	358	≤ 0.02	30	≤ 0.02	–	0
beasley6	15	7.8	8	289	289	289	289	≤ 0.02	60	≤ 0.02	–	0
beasley7	8	18.3	8	430	430	430	430	≤ 0.02	30	≤ 0.02	–	0
beasley8	13	8.2	9	834	834	834	834	≤ 0.02	60	≤ 0.02	–	0
beasley9	18	7.4	9	924	924	924	924	0.3	60	≤ 0.02	–	0
beasley10	13	6.8	7	1452	1452	1452	1452	≤ 0.02	60	≤ 0.02	–	0
beasley11	15	9.1	8	1688	1688	1688	1688	≤ 0.02	60	≤ 0.02	–	0
beasley12	22	8.6	11	1865	1865	1865	1865	0.3	60	≤ 0.02	–	0
cgcut1*	16	10.7	8	244	244	244	244	≤ 0.02	60	1.46	0.3	–
cgcut2*	23	14.8	11	2892	2892	2892	2892	1.2	120	531.93	531.93	–
cgcut3*	62	3.9	11	1860	1860	1846	1840	1.6	30	4.58	4.58	–
gcut1*	10	3.82	4	48 368	48 368	48 368	48 368	≤ 0.02	30	0.01	≤ 0.02	–
gcut2	20	4.6	5	59 798	59 798	59 704	59 563	16.6	30	0.22	0.19	–
gcut3*	30	4.6	6	61 275	61 275	61 275	61 275	2.1	30	3.24	2.16	–
gcut4	50	4.3	6	61 380	61 380	61 380	61 380	0.9	30	376.52	346.99	–
gcut5	10	4.6	4	195 582	195 582	195 582	195 582	≤ 0.02	30	0.5	≤ 0.02	–
gcut6	20	4.1	5	236 305	236 305	236 305	236 305	≤ 0.02	30	0.12	0.06	–
gcut7	30	3.7	5	240 143	240 143	240 143	240 143	≤ 0.02	30	1.07	0.22	–
gcut8	50	4.5	5	245 758	245 758	245 758	245 758	0.1	60	168.5	136.71	–
gcut9	10	4.9	5	939 600	939 600	939 600	939 600	≤ 0.02	30	0.08	≤ 0.02	–
gcut10	20	3.7	5	937 349	937 349	937 349	937 349	0.6	30	0.14	≤ 0.02	–
gcut11	30	4.6	6	969 709	969 709	968 582.3	958 442	≤ 0.02	30	16.3	14.76	–
gcut12*	50	4	4	979 521	979 521	978 727.8	976 877	26.2	30	25.39	16.85	–
gcut13*	32	20.1	18	$\geq 8\,408\,316$ $\geq 8\,622\,498$ $\leq 9\,000\,000$	8 691 947	8 637 809.1	8 615 240	239.3	240	1800	1800	–
hadchr3	42	5	4	1178	1086	1086	1086	≤ 0.02	30	≤ 0.02	–	–
hadchr7	7	6.6	5	1865	1865	1865	1865	0.3	60	≤ 0.02	–	–
hadchr8	22	8.6	11	2517	2517	2517	2517	≤ 0.02	30	≤ 0.02	–	–
hadchr11	10	5.6	6	1270	1270	1270	1270	≤ 0.02	30	≤ 0.02	–	–
hadchr12	15	5.3	4	2949	2949	2949	2949	0.4	30	≤ 0.02	–	–
wang20*	15	5.4	3	2726	2716	2712.5	2711	29.2	60	2.72	2.72	0.11
3	62	3.9	11	1860	1860	1846	1840	1.6	30	≤ 0.02	–	–
3s	62	3.9	6	2726	2726	2722.5	2721	8.6	30	≤ 0.02	–	–
a1	62	4.2	11	2020	2020	2004	1960	≤ 0.02	30	≤ 0.02	–	–
a1s	62	4.2	7	2956	2950	2950	2950	18.1	30	≤ 0.02	–	–
a2	53	5.5	11	2615	2615	2594	2545	2.3	30	≤ 0.02	–	–
a2s	53	5.5	7	3535	3535	3517.9	3516	7.6	30	≤ 0.02	–	–
chl2	19	9.1	10	2326	2326	2326	2326	11.9	60	≤ 0.02	–	–
chl2s	19	9.1	9	3336	3336	3336	3336	55.5	60	≤ 0.02	–	–
chl3	35	89.8	35	5283	5283	5283	5283	≤ 0.02	240	≤ 0.02	–	–
chl3s	35	89.8	35	7402	7402	7402	7402	≤ 0.02	240	≤ 0.02	–	–
chl4	27	92.7	27	8998	8858	8763.5	8658	≤ 0.02	240	≤ 0.02	–	–
chl4s	27	92.7	27	13 932	13 932	13 932	13 932	≤ 0.02	240	≤ 0.02	–	–
chl5	18	7.4	5	589	589	587	584	1.7	60	≤ 0.02	–	–
okp1*	50	14.3	9	27 718	27 718	27 542.7	27 486	6.6	120	35.84	11.6	0.05
okp2	30	9.6	11	22 502	22 214	22 098.6	21 947	22.9	60	1559	1535.95	2.14
okp3*	30	8.3	11	24 019	24 019	23 859.8	23 531	11	60	10.63	1.91	3.4
okp4	61	10.1	8	32 893	32 893	32 893	32 893	4.9	60	4.05	2.13	0.66
okp5*	97	12.6	15	27 923	27 923	26 759	25 468	5.4	120	488.27	488.27	≤ 0.02

Table 3
Average results for the 630 `ngcutfs` instances

Instance	Avg. 1D deviation			Time		
	Beasley	Egeblad and Pisinger	Alvarez-Valdes et al.	Beasley	Egeblad and Pisinger	Alvarez-Valdes et al.
Type 1	1.64	1.19	0.95	558.1	37.38	19.61
Type 2	1.70	1.29	1.06	668.4	45.03	23.84
Type 3	1.66	1.19	0.94	830.0	62.05	32.56

bin is always twice its width. The naming convention is `ep2-n-c-t-p`, where $n \in \{30, 50, 100, 200\}$ is the number of rectangles, $c \in (T, W, S, U, D)$ describes the class, $t \in (C, R)$ describes if it is clustered or random, $p \in 25, 75$ describes the size of the bin in percentage of the total rectangle area. The profit of the rectangles is always the area of the rectangle times a random number from $\{1, 2, 3\}$. The instances are presented in Table 4 and are available along with the source code to generate them at this web address: <http://www.diku.dk/~pisinger/codes.html>.

6.2.3. Parameter setting

Three parameters are crucial for the results of simulated annealing: The start time t_0 , the time step t_s and the stopping-criteria. A time limit is used as stopping-criteria. Suitable values of t_0 and t_s and the time limit depend on the complexity of the instance.

For each instance we determine two indicator values n_0 and n_1 . We set $n_0 = n \cdot \frac{V_{\text{knapack}}}{V_{\text{items}}}$, where V_{knapack} is the area (volume) of the knapsack and V_{items} is sum of the items' area (volume). It indicates the average number of items a knapsack may contain.

We use the value n_0 to determine the running time of our experiments: For an instance with n rectangles and n_0 defined as above let $F(n, n_0) = n_0 \lg n$. If we expect n_0 items to fit into the knapsack and the order of the items matters then there are roughly $\frac{n!}{n_0!} \approx n^{n_0}$ ways to select the items and we may expect that there are roughly n^{n_0} different possible solutions to search. Thus $F(n, n_0) = \lg(n^{n_0})$ should give us a rough indication of the size of the solution space of the instance.

The running time $T(n, n_0)$ (in s) of each instance is determined from the value $F(n, n_0)$ as follows:

$$T(n, n_0) = \begin{cases} 30 & \text{for } F(n, n_0) < 25, \\ 60 & \text{for } 25 \leq F(n, n_0) < 65, \\ 120 & \text{for } 65 \leq F(n, n_0) < 100, \\ 240 & \text{for } 100 \leq F(n, n_0) < 250, \\ 600 & \text{for } 250 \leq F(n, n_0). \end{cases} \quad (22)$$

The value n_1 is the number of items chosen in an optimal solution of (1DKP) defined in (9). For all considered instances, this problem is solved to optimum very quickly using the exact method by Pisinger [33]. n_1 reflects the number of rectangles to be expected in an optimal solution which is another indicator of the complexity of the instance.

To determine appropriate values of t_0 and t_s we experimented with the 23 instances marked with “*” in Tables 2 and 4. These contain between 16 and 200 rectangles.

We performed the experiments with $t_0 \in \{10^{-3}, 10^{-2}, 10^{-1}, 10^0, 10^1, 10^2, 10^3, 10^4, 10^5\}$ and $t_s \in \{10^2, 10^1, 10^{-1}, 10^{-3}, 10^{-5}, 10^{-7}, 10^{-9}, 10^{-11}, 10^{-13}\}$. For the 23 instances, each of the 81 combinations of t_0 and t_s were tested using the running times from (22). Results from four selected instances are presented in Fig. 6.

Based on the results of the parameter tuning for the 23 instances, we were able to establish that good values of t_0 and t_s are

$$t_0 = n_1^2, \quad t_s = \frac{n_1^2}{10^7}.$$

The values can be interpreted in the following way: The higher the t_0 and t_s the less likely is the acceptance of a non-improving permutation. The larger the number of rectangles is in an optimal solution the more improving steps must be undertaken before the heuristic reaches a local minimum.

Table 4
Results for the ep2 instances

Instance	n	n_0	n_1	Bound	Best	Best Time	Seed Time	Best/Bound	Avg./Bound	Worst/Bound
ep2-30-D-C-25	30	7.5	5	6339	6160	60	6.2	2.82	2.82	2.82
ep2-30-D-C-75*	30	22.2	25	12 760	12 588	240	12.7	1.35	1.46	1.97
ep2-30-D-R-25	30	7.4	6	6877	6129	60	35.1	10.88	10.88	10.88
ep2-30-D-R-75	30	22.4	24	14 395	14 259	240	148.8	0.97	1.18	1.53
ep2-30-S-C-25	30	7.4	11	82 059	81 944	60	0.5	0.14	0.14	0.14
ep2-30-S-C-75	30	22.4	22	198 013	195 670	240	12.9	1.20	1.37	1.47
ep2-30-S-R-25	30	7.5	9	97 151	85 220	60	54.0	12.28	14.19	14.40
ep2-30-S-R-75	30	22.5	24	228 676	225 747	240	174.8	1.28	1.34	1.69
ep2-30-T-C-25	30	7.5	9	30 462	22 608	60	0.1	25.78	25.78	25.78
ep2-30-T-C-75	30	22.4	22	73 944	73 565	240	186.8	0.54	0.55	0.61
ep2-30-T-R-25*	30	7.4	8	30 570	26 034	60	0.2	14.84	14.84	14.84
ep2-30-T-R-75	30	22.3	23	78 323	77 627	240	206.4	0.89	1.20	1.41
ep2-30-U-C-25	30	7.5	8	143 750	133 101	60	≤ 0.02	7.74	7.74	7.74
ep2-30-U-C-75	30	22.5	22	354 871	343 528	240	23.4	3.22	3.22	3.22
ep2-30-U-R-25	30	7.5	8	143 127	137 739	60	≤ 0.02	3.76	3.76	3.76
ep2-30-U-R-75	30	22.4	22	366 621	352 982	240	0.5	3.74	3.74	3.74
ep2-30-W-C-25	30	10.6	10	35 727	35 727	60	11.8	0.00	0.00	0.00
ep2-30-W-C-75	30	22.4	22	46 176	46 176	240	≤ 0.02	0.00	0.00	0.00
ep2-30-W-R-25	30	10.5	13	34 332	34 332	60	0.2	0.00	0.00	0.00
ep2-30-W-R-75*	30	22.2	23	45 777	45 777	240	0.5	0.00	0.00	0.00
ep2-50-D-C-25	50	12.4	11	11 094	10 369	120	96.8	6.56	6.85	7.21
ep2-50-D-C-75	50	37.1	44	21 433	21 076	240	127.2	1.70	1.96	2.40
ep2-50-D-R-25	50	12.5	19	12 495	11 475	120	119.2	8.16	8.82	12.01
ep2-50-D-R-75	50	37.1	38	31 657	31 426	240	235.6	0.78	1.51	2.11
ep2-50-S-C-25	50	12.5	14	161 376	154 653	120	24.5	4.21	4.21	4.21
ep2-50-S-C-75	50	37.4	38	391 915	387 690	240	40.9	1.08	1.75	3.41
ep2-50-S-R-25	50	12.4	12	142 758	138 263	120	107.4	3.19	3.22	3.27
ep2-50-S-R-75	50	37.3	38	306 187	303 774	240	234.3	0.82	1.03	1.30
ep2-50-T-C-25*	50	12.4	19	46 065	46 019	120	12.3	0.10	0.10	0.10
ep2-50-T-C-75	50	37.2	39	118 094	114 081	240	56.9	3.42	4.25	4.82
ep2-50-T-R-25	50	12.4	12	51 175	51 175	120	7.7	0.00	0.00	0.00
ep2-50-T-R-75	50	37.2	36	144 602	141 056	240	138.8	2.45	2.81	4.02
ep2-50-U-C-25	50	12.5	13	242 937	231 774	120	4.8	4.60	4.79	4.87
ep2-50-U-C-75	50	37.4	37	632 455	619 080	240	207.4	2.17	2.47	3.64
ep2-50-U-R-25	50	12.4	12	263 251	227 979	120	8.5	13.40	13.75	13.90
ep2-50-U-R-75*	50	37.4	38	576 134	564 394	240	235.8	2.07	2.29	3.54
ep2-50-W-C-25*	50	13.7	15	50 130	50 130	120	18.1	0.00	0.00	0.00
ep2-50-W-C-75	50	37.3	37	94 279	87 320	240	21.0	7.38	7.38	7.38
ep2-50-W-R-25	50	12.9	13	55 920	55 920	120	0.5	0.00	0.00	0.00
ep2-50-W-R-75	50	37.4	35	115 156	114 656	240	97.0	0.49	0.95	1.22
ep2-100-D-C-25	100	24.5	19	23 250	22 730	240	225.0	2.25	2.80	3.47
ep2-100-D-C-75	100	74.0	77	51 241	49 732	600	536.7	2.99	3.85	4.99
ep2-100-D-R-25	100	24.7	23	22 326	22 133	240	237.9	0.92	1.31	2.14
ep2-100-D-R-75	100	74.6	73	51 231	50 874	600	520.3	0.73	1.11	2.08
ep2-100-S-C-25	100	24.9	28	323 640	314 198	240	39.2	2.98	3.57	4.15
ep2-100-S-C-75	100	74.9	81	756 554	747 129	600	580.4	1.29	1.63	2.49
ep2-100-S-R-25	100	24.9	27	254 616	250 242	240	210.3	1.77	2.69	3.70
ep2-100-S-R-75*	100	74.9	80	523 573	519 787	600	595.6	0.76	1.26	1.68
ep2-100-T-C-25*	100	24.9	27	92 331	92 331	240	28.2	0.00	0.00	0.00
ep2-100-T-C-75	100	74.7	69	265 970	256 480	600	316.4	3.65	4.06	4.96
ep2-100-T-R-25	100	24.8	25	103 359	102 837	240	34.3	0.56	0.78	1.05
ep2-100-T-R-75	100	74.8	77	262 492	257 927	600	573.4	1.78	2.39	3.49
ep2-100-U-C-25	100	25.0	25	547 224	505 794	240	42.2	7.63	8.70	9.77
ep2-100-U-C-75	100	75.0	75	1 433 510	1 400 642	600	14.0	2.29	2.29	2.29
ep2-100-U-R-25	100	25.0	26	518 661	493 901	240	157.0	4.77	5.28	5.51
ep2-100-U-R-75	100	74.9	75	1 216 431	1 203 544	600	418.4	1.10	1.26	1.74
ep2-100-W-C-25	100	24.9	29	70 437	70 164	240	4.9	0.39	0.71	1.06

Table 4 (Continued)

Instance	n	n_0	n_1	Bound	Best	Best Time	Seed Time	Best/Bound	Avg./Bound	Worst/Bound
ep2-100-W-C-75	100	74.9	88	167 577	1 599 29	600	450.9	4.56	4.88	6.38
ep2-100-W-R-25*	100	25.0	27	70 224	70 224	240	165.3	0.00	0.04	0.12
ep2-100-W-R-75	100	74.7	77	247 494	230 809	600	393.8	6.80	7.40	8.33
ep2-200-D-C-25*	200	49.2	63	46 728	45 987	600	530.4	1.73	2.55	3.99
ep2-200-D-C-75	200	149.7	160	1 278 34	1 241 46	600	536.5	2.99	3.87	4.73
ep2-200-D-R-25	200	49.6	57	43 605	42 138	600	591.4	3.46	4.83	5.80
ep2-200-D-R-75	200	149.3	154	99 002	97 694	600	450.6	1.39	2.19	3.47
ep2-200-S-C-25	200	49.9	54	649 446	636 050	600	39.9	2.15	2.78	3.43
ep2-200-S-C-75	200	149.8	155	1 315 780	1 297 053	600	570.2	1.49	2.29	3.08
ep2-200-S-R-25	200	49.8	48	519 498	506 437	600	533.1	2.59	5.30	7.48
ep2-200-S-R-75*	200	149.7	143	1 225 926	1 201 303	600	581.5	2.06	3.10	3.75
ep2-200-T-C-25*	200	50.0	41	188 684	184 528	600	319.9	2.26	3.48	3.84
ep2-200-T-C-75	200	149.5	152	441 796	431 290	600	558.1	2.44	3.35	4.57
ep2-200-T-R-25	200	49.7	50	190 638	188 967	600	587.1	0.98	1.23	1.53
ep2-200-T-R-75	200	149.5	145	476 289	468 381	600	597.6	1.72	2.67	3.64
ep2-200-U-C-25	200	50.0	51	1 084 836	1 056 636	600	512.6	2.84	2.96	2.97
ep2-200-U-C-75	200	149.8	151	2 313 551	2 265 176	600	549.6	2.15	2.58	2.78
ep2-200-U-R-25	200	49.9	49	1 039 584	1 015 452	600	587.0	2.39	3.70	5.05
ep2-200-U-R-75	200	149.7	149	2 447 655	2 400 803	600	176.6	1.97	2.14	2.65
ep2-200-W-C-25	200	49.6	76	161 002	157 508	600	577.0	2.21	2.97	3.83
ep2-200-W-C-75	200	149.4	160	390 001	375 767	600	566.1	3.78	4.26	5.10
ep2-200-W-R-25	200	49.9	50	196 128	196 086	600	170.7	0.02	0.14	0.26
ep2-200-W-R-75	200	149.6	146	511 386	503 222	600	586.7	1.69	2.24	2.79

6.2.4. Results

Using the parameters of the previous section, our heuristic was applied to the instances described in Sections 6.2.1 and 6.2.2. To determine the robustness of the heuristic, we ran it on each instance with 10 different random seeds. For the classical instances, the optimal solution value is reported [9] where known. For the remaining instances, we compare our results with the upper bounds described in Section 6.1.

The results are reported in Tables 2–4 which all follow the same format. The average, best and worst results of our heuristic on each instance for the 10 seeds are reported in the columns entitled ‘Avg.’, ‘Best’ and ‘Worst’, respectively. The time before the heuristic discovered the best solution is reported in the column entitled ‘Best Time’ and the time spent on each seed is reported in the column entitled ‘Seed Time’ (total time = 10 · Seed Time).

In Table 2, we report results for the classical instances. The column ‘Optimal’ contains values of optimal solutions. For `gcut13`, no optimal value is currently known, but we have reported the results and upper bounds of Caprara and Monaci [11] and Fekete and Schepers [9], respectively. The ‘Exact Methods Time’ represent the running time of the algorithms by Caprara and Monaci [11] and Fekete and Schepers [9]. All running times are in seconds. The running times of Alvarez-Valdes et al. [20] are reported in the column entitled ‘Alvarez-Valdes’ (instances were solved to optimality). Instances marked with ‘*’ were used for parameter tuning.

The heuristic finds the optimal value in all but four of the instances. For the remaining instances except `hadchr3`, the deviation is less than 2% from optimum. The heuristic is able to improve the best known solution of `gcut13` by 0.8%. The time to find the best solution is generally below 1 s for the small instances and reaches only a few minutes for the most difficult instances.

For the constrained guillotine-cut instances, we generally get equal or better results than the original authors since we also allow for non-guillotine packing. However, the unconstrained instances (`gcut`) have been transformed and are not comparable with results from the original paper.

Table 3 summarizes results on the 630 `ngcuts` instances as well as the results of Beasley [18] and Alvarez-Valdes et al. [20,19]. As in [18,20,19] results are reported as the average *percent* deviation from the (1DKP) upper bound in the columns marked ‘Avg./1D’. The average time in seconds it took to reach the best solution is given in columns marked ‘Time’. The results in ‘Beasley’ are taken from [18] and the results in ‘Alvarez-Valdes et al.’ are taken from [20].

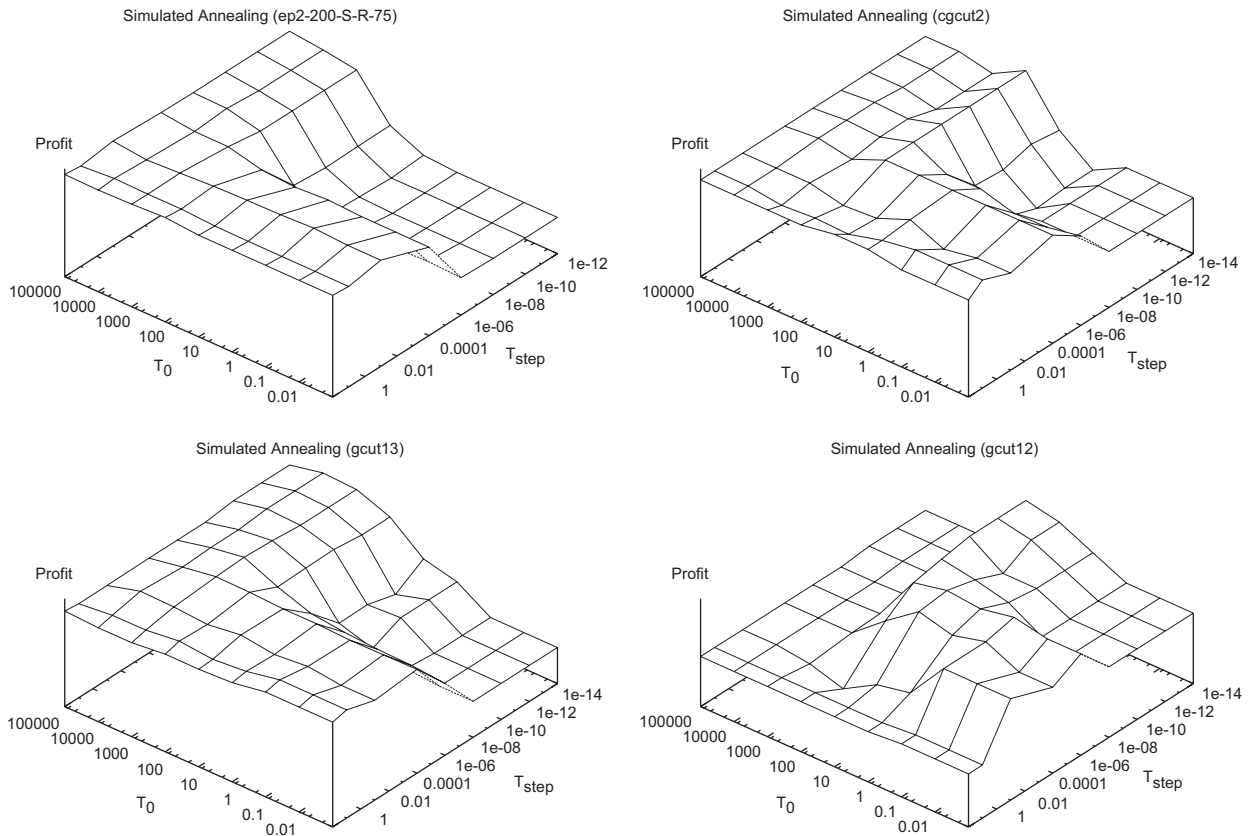


Fig. 6. Results of the simulated annealing heuristic for different values of t_0 and t_s on four different instances.

It is seen that in terms of solution quality, our results are 0.4% better than those of Beasley and only 0.25% worse than Alvarez-Valdes et al. Since their reported results for large instances are found within the first second, we suspect that the approach by Alvarez-Valdes et al. benefits from a greedy constructive algorithm that is applied before their local search, which seems to find optimal solutions for instances with hundreds of items or more. In the present paper, we are mainly interested in comparing the local search frameworks and not the initial greedy heuristics. Moreover, it should be emphasized that although the *ngcutfs* instances contain up to 4000 items, the average value of n_0 and n_1 are 5.4 and 7.9 and the maximal values are 9.31 and 21, respectively. This indicates that the number of rectangles which can fit inside the knapsack is only a few dozen. Therefore, it would be interesting to compare results for instances where more than 100 items fit simultaneously inside the knapsack.

The results on the 80 newly proposed benchmarks are listed in Table 4. Here ‘Bound’ refers to the upper bound based on conservative scales and ‘Best/Bound’, ‘Avg./Bound’, ‘Worst/Bound’ are the percentage deviations between the heuristic best, average and worst solutions and the conservative scale upper bound calculated as $100 - [\text{solution value}]/[\text{boundvalue}] \cdot 100$. Only for 11 of the instances is the best result more than 5% from the upper bound and on average the deviation is only 3.0%. The average deviation is 4.6%, 3.1%, 2.4% and 2.1% for $n = 30, 50, 100$ and 200, respectively. For nine of the instances, the heuristic finds the optimal solution since the deviation from the upper bound is 0. This shows the heuristic’s ability to find good solutions for both small and large instances and that the gap between solution value and upper bound decreases as the number of items and size of the knapsack increases.

Fig. 7 illustrates the behavior of the heuristic over time for the instances *okp5*, *ep-100-S-R-75* and *ep2-200-U-C-75*. The y-axis is the percentage of the best solution found during the run and the x-axis is the percentage of the full running time for each instance. The graph shows that after roughly half the running time the solution value stays within 1% of the best found solution value and that the heuristic quickly converges but allows for minor changes throughout the entire solution process. The best results for three of the instances are shown in Fig. 8.

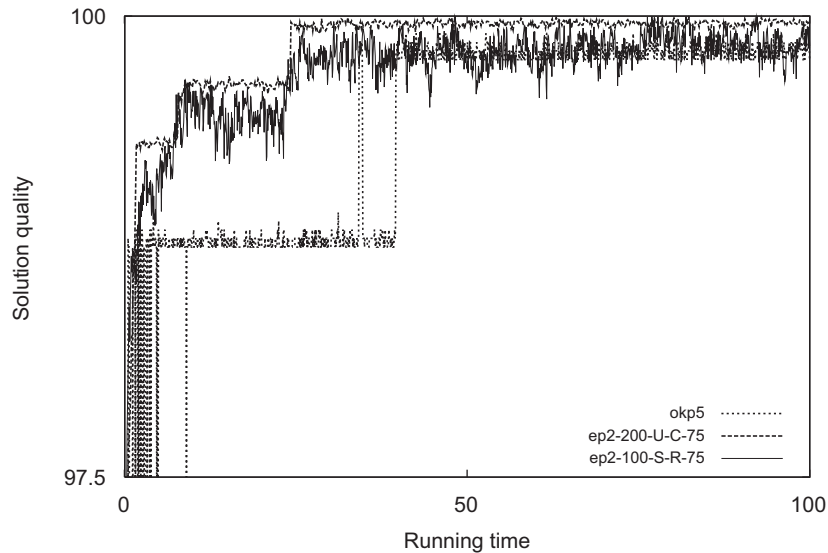


Fig. 7. Illustration of the heuristic behavior over time.

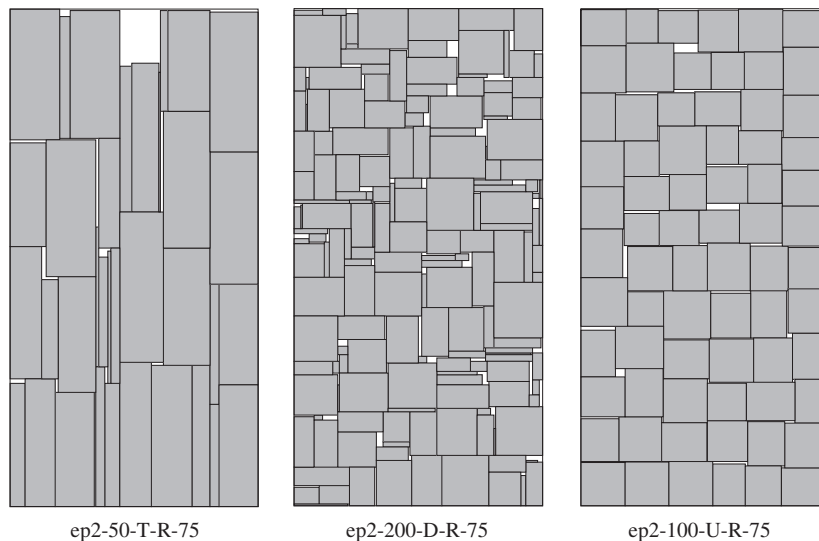


Fig. 8. Best results for three two-dimensional instances without rotation.

6.2.5. Results with rotations

We repeated all tests allowing rotation, doubling the running time to accommodate for the larger solution space. A maximum time limit of 600 s was still assigned to all instances. Parameter tuning revealed that the setting of t_0 and t_s reported in Section 6.2.3 also give good results when rotation is allowed.

The results on the two sets of instances are reported in Tables 5 and 6. The tables follow the same format as the tables without rotation, however the column ‘No rotation’ is the value of the optimal solution without rotation for the classical instances and the best solution without rotation for the ep2 instances where the optimal solutions are not known. For *gcut13* no optimal values without rotation is known and we have reported the best solution from Table 2. To compare the quality of the solution we used the (1DKP) upper bound, which is weaker than the bound from conservative scales. The column ‘No rotation/1D’ contains percentage deviations between the result without rotation and the (1DKP) upper

Table 5

Results with rotation for the benchmark instances

Instance	1D	No rotation	Best	Best Time	Seed Time	No rotation/1D	Best/1D	Avg./1D	Worst/1D
beasley1	201	164	193	≤0.02	60	18.41	3.98	3.98	3.98
beasley2	253	230	250	0.3	120	9.09	1.19	1.19	1.19
beasley3	266	247	259	0.4	120	7.14	2.63	2.63	2.63
beasley4	275	268	268	≤0.02	60	2.55	2.55	2.55	2.55
beasley5	373	358	370	≤0.02	60	4.02	0.80	0.80	0.80
beasley6	317	289	300	23.9	120	8.83	5.36	5.99	5.99
beasley7	430	430	430	≤0.02	60	0.00	0.00	0.00	0.00
beasley8	938	834	886	0.2	120	11.09	5.54	5.54	5.54
beasley9	962	924	924	4.4	120	3.95	3.95	4.57	4.57
beasley10	1517	1452	1452	≤0.02	120	4.28	4.28	4.28	4.28
beasley11	1864	1688	1786	≤0.02	120	9.44	4.18	4.18	4.18
beasley12	2012	1865	1932	1.8	120	7.31	3.98	4.47	4.47
cgcut1	260	244	260	0.7	120	6.15	0.00	0.00	0.00
cgcut2	2919	2892	2909	115.3	240	0.92	0.34	0.34	0.34
cgcut3	2020	1860	1940	6.3	60	7.92	3.96	3.96	3.96
gcut1	62 488	48 368	58 136	≤0.02	60	22.60	6.96	6.96	6.96
gcut2	62 500	59 798	60 656	23.9	60	4.32	2.95	3.22	3.31
gcut3	62 500	61 275	61 275	49.6	60	1.96	1.96	2.26	2.92
gcut4	62 500	61 380	61 710	30.1	60	1.79	1.26	1.28	1.47
gcut5	249 854	195 582	233 969	1.1	60	21.72	6.36	6.36	6.36
gcut6	249 992	236 305	239 467	0.1	60	5.47	4.21	4.21	4.21
gcut7	249 998	240 143	245306	34.5	60	3.94	1.88	2.64	2.83
gcut8	250 000	245 758	247 462	21	120	1.70	1.02	1.12	1.26
gcut9	997 256	939 600	953 293	≤0.02	60	5.78	4.41	4.41	4.41
gcut10	999 918	937 349	938 036	0.1	60	6.26	6.19	6.19	6.19
gcut11	1 000 000	969 709	979 580	20	60	3.03	2.04	2.46	2.96
gcut12	1 000 000	979 521	987 674	8.9	60	2.05	1.23	1.23	1.23
gcut13	9 000 000	≥8 736 757	8 897 979	344.5	480	2.92	1.13	1.64	2.32
hadchr-3	1347	1178	1272	≤0.02	60	12.55	5.57	5.57	5.57
hadchr-7	2012	1865	1932	9.7	120	7.31	3.98	4.03	4.47
hadchr-8	3079	2517	2722	≤0.02	60	18.25	11.59	11.59	11.59
hadchr-11	1547	1270	1431	1	60	17.91	7.50	7.50	7.50
hadchr-12	3604	2949	3252	9.8	60	18.17	9.77	9.77	9.77
wang20	2800	2771	2762	118	120	1.04	1.36	1.50	1.50
3	2020	1860	1940	0.9	60	7.92	3.96	3.96	3.96
3s	2800	2726	2758	12.6	60	2.64	1.50	1.54	1.57
a1	2140	2020	2120	40.2	60	5.61	0.93	1.78	1.87
a1s	3000	2956	2985	1.7	60	1.47	0.50	0.50	0.50
a2	2705	2615	2690	18.4	120	3.33	0.55	0.55	0.55
a2s	3600	3335	3579	99.1	120	7.36	0.58	0.58	0.58
chl2	2502	2326	2429	8.9	120	7.03	2.92	3.20	4.32
chl2s	3410	3336	3390	7.8	120	2.17	0.59	0.59	0.59
chl3	5283	5283	5283	≤0.02	480	0.00	0.00	0.00	0.00
chl3s	7402	7402	7402	≤0.02	480	0.00	0.00	0.00	0.00
chl4	8998	8998	8912	≤0.02	480	0.00	0.96	1.77	2.98
chl4s	13 932	13 932	13 932	≤0.02	480	0.00	0.00	0.00	0.00
chl5	600	589	600	7.6	120	1.83	0.00	0.00	0.00
okp1	29 133	27 718	28 423	41.7	240	4.86	2.44	4.24	5.47
okp2	24 800	22 502	24 263	1.6	120	9.27	2.17	5.19	8.05
okp3	26 714	24 019	25 216	7.2	120	10.09	5.61	7.28	10.99
okp4	33 631	32 893	32 893	19.7	120	2.19	2.19	2.19	2.19
okp5	29 045	27 923	27 971	121	240	3.86	3.70	6.06	11.43

Table 6
Results with rotation for the ep2 instances

Instance	Best	Best Time	Seed Time	No rotation/1D	Best/1D	Avg./1D	Worst/1D
ep2-30-D-C-25	6340	57.4	120	4.23	1.43	1.52	1.55
ep2-30-D-C-75	12 672	347.5	480	1.35	0.69	1.20	1.50
ep2-30-D-R-25	6875	113.6	120	16.54	6.39	6.58	6.66
ep2-30-D-R-75	14 330	115.3	480	0.94	0.45	0.62	0.93
ep2-30-S-C-25	81 944	1.8	120	6.04	6.04	6.04	6.04
ep2-30-S-C-75	195 670	30.4	480	1.18	1.18	1.56	2.20
ep2-30-S-R-25	83 160	1.4	120	12.28	14.40	14.40	14.40
ep2-30-S-R-75	225 747	127.2	480	1.28	1.28	1.40	2.02
ep2-30-T-C-25	26 436	0.5	120	29.29	17.32	17.87	18.52
ep2-30-T-C-75	73 565	144.4	480	0.51	0.51	1.04	1.50
ep2-30-T-R-25	29 535	3.3	120	16.28	5.02	5.02	5.02
ep2-30-T-R-75	77 387	77.6	480	0.89	1.20	1.80	2.49
ep2-30-U-C-25	141 588	0.2	120	7.41	1.50	1.50	1.50
ep2-30-U-C-75	351 682	204.7	480	3.20	0.90	0.90	0.90
ep2-30-U-R-25	140 287	22.6	120	7.51	5.80	5.80	5.80
ep2-30-U-R-75	360 202	450.9	480	3.72	1.75	1.75	1.75
ep2-30-W-C-25	37 851	75.6	120	11.44	6.17	6.33	6.34
ep2-30-W-C-75	53 668	174.6	480	14.33	0.43	0.83	1.37
ep2-30-W-R-25	37 258	32.5	120	9.43	1.71	2.29	2.85
ep2-30-W-R-75	52 895	254.1	480	14.61	1.33	1.80	2.70
ep2-50-D-C-25	11 010	90.0	240	6.54	0.76	1.27	1.66
ep2-50-D-C-75	21 230	278.8	480	1.67	0.95	1.56	2.05
ep2-50-D-R-25	12 456	209.6	240	8.81	1.02	2.78	5.65
ep2-50-D-R-75	31 531	299.1	480	0.73	0.40	1.22	1.84
ep2-50-S-C-25	15 4653	9.3	240	4.17	4.17	4.20	4.43
ep2-50-S-C-75	38 7526	285.3	480	1.08	1.12	2.18	3.41
ep2-50-S-R-25	138 335	235.4	240	3.15	3.10	3.18	3.53
ep2-50-S-R-75	303 605	313.8	480	0.79	0.84	1.23	1.78
ep2-50-T-C-25	46 327	91.8	240	2.26	1.61	1.88	3.42
ep2-50-T-C-75	117 226	303.3	480	3.40	0.74	1.01	1.23
ep2-50-T-R-25	53 383	172.2	240	5.99	1.93	2.10	2.28
ep2-50-T-R-75	142 124	388.1	480	2.45	1.71	2.23	3.56
ep2-50-U-C-25	246 645	0.1	240	14.86	9.39	9.39	9.39
ep2-50-U-C-75	626 218	365.4	480	2.11	0.99	1.46	2.11
ep2-50-U-R-25	241 032	26.6	240	14.39	9.49	9.49	9.49
ep2-50-U-R-75	570 751	398.3	480	2.04	0.93	1.71	1.80
ep2-50-W-C-25	51 360	22.2	240	4.21	1.86	1.86	1.86
ep2-50-W-C-75	111 307	354.7	480	22.17	0.79	1.21	2.05
ep2-50-W-R-25	57 561	102.5	240	3.92	1.10	1.25	1.26
ep2-50-W-R-75	113 868	452.1	480	0.43	1.12	1.55	2.18
ep2-100-D-C-25	23 220	415.3	480	2.24	0.13	0.26	0.71
ep2-100-D-C-75	50 515	378.0	600	2.94	1.42	2.67	4.17
ep2-100-D-R-25	22 281	51.7	480	0.86	0.20	0.42	0.64
ep2-100-D-R-75	50 772	410.8	600	0.70	0.90	1.78	2.81
ep2-100-S-C-25	316 238	341.7	480	2.92	2.29	3.37	4.36
ep2-100-S-C-75	745 608	354.1	600	1.25	1.45	1.91	2.45
ep2-100-S-R-25	251 303	343.7	480	1.72	1.30	2.30	3.79
ep2-100-S-R-75	515 379	509.9	600	0.72	1.57	2.02	3.96
ep2-100-T-C-25	99 429	98.6	480	8.94	1.94	3.31	6.41
ep2-100-T-C-75	260 968	574.9	600	3.57	1.88	2.94	3.77
ep2-100-T-R-25	102 957	303.1	480	0.51	0.39	1.06	2.93
ep2-100-T-R-75	259 019	497.6	600	1.74	1.32	2.30	3.44
ep2-100-U-C-25	531 585	192.6	480	7.57	2.86	5.11	6.17
ep2-100-U-C-75	1 400 642	147.6	600	2.29	2.29	2.37	3.06
ep2-100-U-R-25	504 801	48.2	480	4.77	2.67	4.27	4.95
ep2-100-U-R-75	1 203 634	576.3	600	1.06	1.05	1.63	1.69
ep2-100-W-C-25	94 098	205.3	480	26.63	1.61	2.57	4.24

Table 6 (Continued)

Instance	Best	Best Time	Seed Time	No rotation/1D	Best/1D	Avg./1D	Worst/1D
ep2-100-W-C-75	165 709	599.5	600	6.77	3.40	3.69	5.00
ep2-100-W-R-25	103 431	444.4	480	32.83	1.06	2.83	4.78
ep2-100-W-R-75	242 357	356.2	600	6.74	2.08	2.87	3.49
ep2-200-D-C-25	46 056	473.0	600	1.59	1.44	2.32	3.09
ep2-200-D-C-75	123 724	561.3	600	2.88	3.22	3.56	4.07
ep2-200-D-R-25	42 844	572.0	600	3.36	1.75	3.62	5.14
ep2-200-D-R-75	96 344	593.3	600	1.32	2.68	3.64	5.88
ep2-200-S-C-25	637 579	333.6	600	2.06	1.83	2.86	3.45
ep2-200-S-C-75	1 292 256	596.0	600	1.42	1.79	2.52	4.30
ep2-200-S-R-25	505 687	589.1	600	2.51	2.66	4.78	7.11
ep2-200-S-R-75	1 198 449	576.0	600	2.01	2.24	4.33	6.11
ep2-200-T-C-25	186 248	77.0	600	2.20	1.29	2.32	3.81
ep2-200-T-C-75	432 747	563.5	600	2.38	2.05	2.66	3.54
ep2-200-T-R-25	189 141	577.5	600	0.88	0.79	1.43	2.61
ep2-200-T-R-75	465 169	583.1	600	1.66	2.33	3.67	4.86
ep2-200-U-C-25	1 073 880	238.0	600	2.60	1.01	2.04	2.73
ep2-200-U-C-75	2 265 346	570.2	600	2.09	2.08	2.34	2.71
ep2-200-U-R-25	1 035 978	422.7	600	2.32	0.35	1.95	4.37
ep2-200-U-R-75	2 406 562	591.2	600	1.91	1.68	2.06	2.25
ep2-200-W-C-25	158 468	506.2	600	2.17	1.57	2.36	3.75
ep2-200-W-C-75	382 662	587.2	600	3.65	1.88	3.85	6.30
ep2-200-W-R-25	210 864	575.9	600	8.51	1.62	2.10	2.50
ep2-200-W-R-75	501 738	599.7	600	1.60	1.89	3.51	4.52

bound and ‘Best/1D’, ‘Avg./1D’, ‘Worst/1D’ are deviation between best, average and worst results over the 10 runs and the (1DKP) upper bound.

With the exception of two instances the results with rotation for the classical benchmark instances are better or equal to the results without rotation. This was expected since rotations make it possible to arrange the rectangles in more ways, and hence denser packings can be obtained. Indeed, the heuristic solution is generally larger than 95% of the (1DKP) upper bound, and often reaches 98% when rotation is allowed.

For the 28 of the ep2 instances, we get results which are more than 2% closer to the upper bound than without rotation. We get slightly worse results when allowing rotation in 15 of the 80 cases but only for three instances is the result more than 2% further from the upper bound. This demonstrates the heuristics ability to handle rotation well, even though the solution space is increased by a factor of 2^n which makes it much harder to find near-optimal solutions. The average deviation between the best result and the (1DKP) upper bound for the ep2 instances is 2.3% which means the heuristic performs well even with rotation for large instances. It also shows that utilization increases by a few percent if rotation is allowed.

6.2.6. The instance gcut13

Since gcut13 is the only instance of the classical benchmark instances from the literature where the optimal solution is unknown, we decided to investigate this instance further without considering rotations. We used 144 seeds with 10 min running time on each seed; thus the total running time was 24 h for this instance. The parameters for the simulated annealing were based on the results we gathered during parameter tuning for gcut13, and were set to $t_0 = 0.1$ and $t_s = 10$. The best result was reached after 367 s for one of the seeds and was 8 736 757 which is 0.5% closer to the upper bound than the 8 691 947 we were able to reach with 10 seeds and 4 min running time. This demonstrates that the heuristic is able to return marginally improved results given more running time. The resulting placement can be seen in Fig. 9.

6.3. 3D computational experiments

A set of experiments were also conducted using the three-dimensional variant of our heuristic which follows the same scheme as the experiments conducted for the two-dimensional variant. New instances for 3DKP

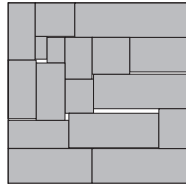


Fig. 9. A solution to `gcut13` with profit 8736757 which was reached after one hundred and forty-four 10 min runs.

Table 7

The five different classes of new `ep3` instances

Class	Description	Width	Height	Depth
F	Flat. Boxes are flat	[50, 100]	[50, 100]	[25, 60]
W	Long. Boxes are long	$\left[1, \frac{2}{3} \cdot 100\right]$	$\left[1, \frac{2}{3} \cdot 100\right]$	[50, 100]
S	Cubes. Boxes are cubes	[1, 100]	Equal to width	Equal to width
U	Uniform. Largest dimension is no more than 200% of the smallest	[50, 100]	[50, 100]	[50, 100]
D	Diverse. Largest dimension can be up-to 50 times the smallest	[1, 50]	[1, 50]	[1, 50]

are introduced in Section 6.3.1, the parameter tuning is outlined in Section 6.3.2 and results are presented in Section 6.3.3.

6.3.1. New instances

As we were unable to locate any benchmark instances for the three-dimensional knapsack problem from the literature, we have generated 60 random instances. It should be noted that Fekete et al. [9] report results for a number of 3DKP problem instances, but the instances are not described in detail. The new instances contain 20, 40 or 60 boxes. The dimensions of the boxes were chosen from five different classes described in Table 7. The width, height and depth of the boxes in each class are selected randomly from the intervals in the ‘Width’, ‘Height’ and ‘Depth’ columns of the table. As for the two-dimensional case, boxes are clustered and random, and the container has a volume equal to 50% or 90% of the total volume of the boxes. The naming convention is `ep3-n-c-t-p`, where $n \in \{20, 40, 60\}$ is the number of boxes, $c \in \{F, L, C, U, D\}$ describes the class, $t \in \{C, R\}$ describes if it is clustered or random, $p \in \{50, 90\}$ describes the size of the bin in percentage of the total box volume. The profit of a box is set to the volume times a random number from $\{1, 2, 3\}$. The instances are presented in Table 8 and are available along with the source code to generate them at this web address: <http://www.diku.dk/~pisinger/codes.html>.

6.3.2. Parameters

As for the two-dimensional instances we determine values n_0 and n_1 and for each instance we set the running time based on the function $F(n, n_0) = n_0 \lg n$, so that for $F(n, n_0) \leq 110$ the running time is set to 120 s, for $110 < F(n, n_0) \leq 200$ the running time is set to 300 s and for $F(n, n_0) > 200$ the running time is set to 600 s.

Nine three-dimensional instances were selected for parameter tuning tests. Values for t_0 was selected from $\{10^0, 10^1, 10^2, 10^3, 10^4, 10^5, 10^7, 10^8\}$ and t_s from $\{10^{-10}, 10^{-8}, 10^{-6}, 10^{-4}, 10^{-1}, 10^0, 10^2, 10^4, 10^6\}$. Based on the 81 parameter combinations we found results similar to the two-dimensional case, and based on these we determined good values to be

$$t_0 = \frac{n_1^2}{5} \quad \text{and} \quad t_s = n_1^2.$$

6.3.3. Results

The results from our 3D tests are presented in Table 8 using the same format as the tables of the two-dimensional results.

For the instances with 20 items, the gap between the best found solution and the conservative scales upper bound is as large as 38.7% for one instance but only 13.6% on average and in two cases the heuristic is able to reach the upper

Table 8
Results for the new ep3 instances

Instance	n	n_0	n_1	ID	Bound	Best	Best Time	Seed Time	Best/Bound	Avg./Bound	Worst/Bound
ep3-20-C-C-50.3kp	20	9.9	13	125 170	65 308	65 308	≤ 0.02	120	0.00	0.00	0.00
ep3-20-C-C-90.3kp	20	17.8	19	155 222	97 422	80 124	≤ 0.02	120	17.76	17.76	17.76
ep3-20-C-R-50.3kp	20	9.9	15	108 421	63 849	62 364	≤ 0.02	120	2.33	2.33	2.33
ep3-20-C-R-90.3kp	20	17.8	19	118 927	80 956	66 844	≤ 0.02	120	17.43	17.43	17.43
ep3-20-D-C-50.3kp	20	9.8	13	21 792	13 192	13 192	≤ 0.02	120	0.00	0.00	0.00
ep3-20-D-C-90.3kp	20	17.6	17	38 842	30 728	27 848	0.1	120	9.37	9.37	9.37
ep3-20-D-R-50.3kp	20	10.0	12	19 298	17 845	15 170	23.6	120	14.99	16.26	18.08
ep3-20-D-R-90.3kp	20	17.8	18	25 676	24 836	20 822	3.1	120	16.16	16.16	16.16
ep3-20-F-C-50.3kp	20	9.9	13	114 718	89 771	71 816	≤ 0.02	120	20.00	20.00	20.00
ep3-20-F-C-90.3kp	20	17.8	17	246 440	184 773	167 281	0.6	120	9.47	12.46	13.75
ep3-20-F-R-50.3kp	20	9.9	11	128 928	85 712	80 962	0.1	120	5.54	5.54	5.54
ep3-20-F-R-90.3kp	20	18.0	18	191 008	172 182	155 155	0.4	120	9.89	10.35	12.03
ep3-20-L-C-50.3kp	20	10.0	8	33 780	28 716	20 835	23.0	120	27.44	27.65	27.67
ep3-20-L-C-90.3kp	20	17.8	18	63 772	51 988	48 404	≤ 0.02	120	6.89	6.89	6.89
ep3-20-L-R-50.3kp	20	9.9	10	31 415	30 037	24 809	0.6	120	17.41	17.55	18.15
ep3-20-L-R-90.3kp	20	18.0	17	37 570	37 832	32 892	29.8	120	13.06	14.98	16.80
ep3-20-U-C-50.3kp	20	9.9	10	154 428	148 412	91 036	26.6	120	38.66	41.08	42.11
ep3-20-U-C-90.3kp	20	17.9	17	162 251	152 837	132 291	0.2	120	13.44	16.15	17.04
ep3-20-U-R-50.3kp	20	9.9	10	140 762	123 675	97 358	0.2	120	21.28	22.66	28.45
ep3-20-U-R-90.3kp	20	17.9	18	211 545	209 795	188 691	26.3	120	10.06	10.06	10.06
ep3-40-C-C-50.3kp	40	19.7	29	22 4754	170 318	141 418	0.1	120	16.97	16.97	16.97
ep3-40-C-C-90.3kp	40	35.7	33	358 993	252 425	243 447	0.6	300	3.56	3.56	3.56
ep3-40-C-R-50.3kp	40	19.8	22	202 725	145 067	126 069	0.6	120	13.10	13.10	13.10
ep3-40-C-R-90.3kp	40	35.9	38	245 888	226 890	218 971	11.0	300	3.49	3.49	3.49
ep3-40-D-C-50.3kp	40	19.9	22	34 128	32 520	20 464	0.1	120	37.07	40.42	46.33
ep3-40-D-C-90.3kp	40	35.6	30	70 302	63 600	56 124	113.9	300	11.75	14.08	14.34
ep3-40-D-R-50.3kp	40	19.2	24	35 001	35 803	28 019	36.2	120	21.74	24.17	26.14
ep3-40-D-R-90.3kp	40	35.7	36	52 769	53 698	49 476	244.0	300	7.86	9.11	9.81
ep3-40-F-C-50.3kp	40	19.9	19	225 834	180 302	161 310	23.3	120	10.53	10.53	10.53
ep3-40-F-C-90.3kp	40	35.8	33	543 292	473 556	416 467	3.4	300	12.06	13.36	14.06
ep3-40-F-R-50.3kp	40	19.7	21	312 163	277 727	238 990	93.7	120	13.95	17.36	22.64
ep3-40-F-R-90.3kp	40	35.7	37	396 059	400 886	367 045	282.3	300	8.44	8.78	9.60
ep3-40-L-C-50.3kp	40	19.6	21	80 264	61 222	56 742	76.9	120	7.32	8.63	8.78
ep3-40-L-C-90.3kp	40	35.5	34	117 699	115 632	106 602	223.6	300	7.81	8.71	9.90
ep3-40-L-R-50.3kp	40	19.9	21	61 275	62 931	50 389	80.9	120	19.93	21.93	26.22
ep3-40-L-R-90.3kp	40	35.6	35	85 517	87 114	77 514	235.2	300	11.02	11.97	13.10
ep3-40-U-C-50.3kp	40	20.0	20	216 956	185 028	164 976	0.4	120	10.84	11.87	14.27
ep3-40-U-C-90.3kp	40	35.9	36	461 920	442 120	388 456	78.5	300	12.14	12.14	12.14
ep3-40-U-R-50.3kp	40	19.8	20	271 163	276 228	242 819	12.0	120	12.09	12.86	14.07
ep3-40-U-R-90.3kp	40	35.9	37	401 482	408 182	371 662	286.9	300	8.95	9.58	10.52
ep3-60-C-C-50.3kp	60	29.7	37	363 614	373 341	252 584	3.1	300	32.34	34.40	34.98
ep3-60-C-C-90.3kp	60	53.7	57	620 658	534 168	499 572	92.8	600	6.48	6.48	6.48
ep3-60-C-R-50.3kp	60	29.8	44	303 842	229 567	224 875	273.2	300	2.04	4.57	16.73
ep3-60-C-R-90.3kp	60	53.7	58	427 077	415 339	384 302	49.4	600	7.47	7.80	7.97
ep3-60-D-C-50.3kp	60	29.2	38	90 084	75 732	65 502	63.4	300	13.51	13.57	13.63
ep3-60-D-C-90.3kp	60	53.4	55	129 504	115 968	105 540	450.7	600	8.99	9.01	9.02
ep3-60-D-R-50.3kp	60	29.4	41	56 227	58 185	49 011	227.3	300	15.77	18.60	20.39
ep3-60-D-R-90.3kp	60	53.4	56	73 109	74 464	67 202	337.1	600	9.75	10.21	10.70
ep3-60-F-C-50.3kp	60	29.7	40	522 640	517 890	467 730	3.9	300	9.69	9.69	9.69
ep3-60-F-C-90.3kp	60	53.3	50	642 192	616 504	540 732	145.3	600	12.29	12.92	14.60
ep3-60-F-R-50.3kp	60	29.6	30	437 008	442 187	378 079	138.6	300	14.50	15.66	18.52
ep3-60-F-R-90.3kp	60	53.6	55	668 571	685 430	625 362	413.5	600	8.76	9.65	10.02
ep3-60-L-C-50.3kp	60	29.6	34	131 466	127 740	119 676	229.3	300	6.31	9.64	11.53
ep3-60-L-C-90.3kp	60	53.5	52	124 652	127 340	113 107	226.8	600	11.18	12.09	13.08
ep3-60-L-R-50.3kp	60	29.3	29	74 166	76 856	62 190	297.8	300	19.08	21.03	23.27
ep3-60-L-R-90.3kp	60	53.3	54	117 361	120 313	107 270	373.4	600	10.84	11.67	12.06
ep3-60-U-C-50.3kp	60	29.6	34	430 130	441 418	361 340	42.6	300	18.14	18.83	20.13

Table 8 (Continued)

Instance	<i>n</i>	<i>n</i> ₀	<i>n</i> ₁	1D	Bound	Best	Best Time	Seed Time	Best/Bound	Avg./Bound	Worst/Bound
ep3-60-U-C-90.3kp	60	54.0	54	324 792	343 738	293 852	181.7	600	14.51	15.57	16.70
ep3-60-U-R-50.3kp	60	29.7	31	452 615	465 935	396 794	281.2	300	14.84	16.07	17.17
ep3-60-U-R-90.3kp	60	53.6	55	604 852	618 814	550 375	531.6	600	11.06	11.67	12.17

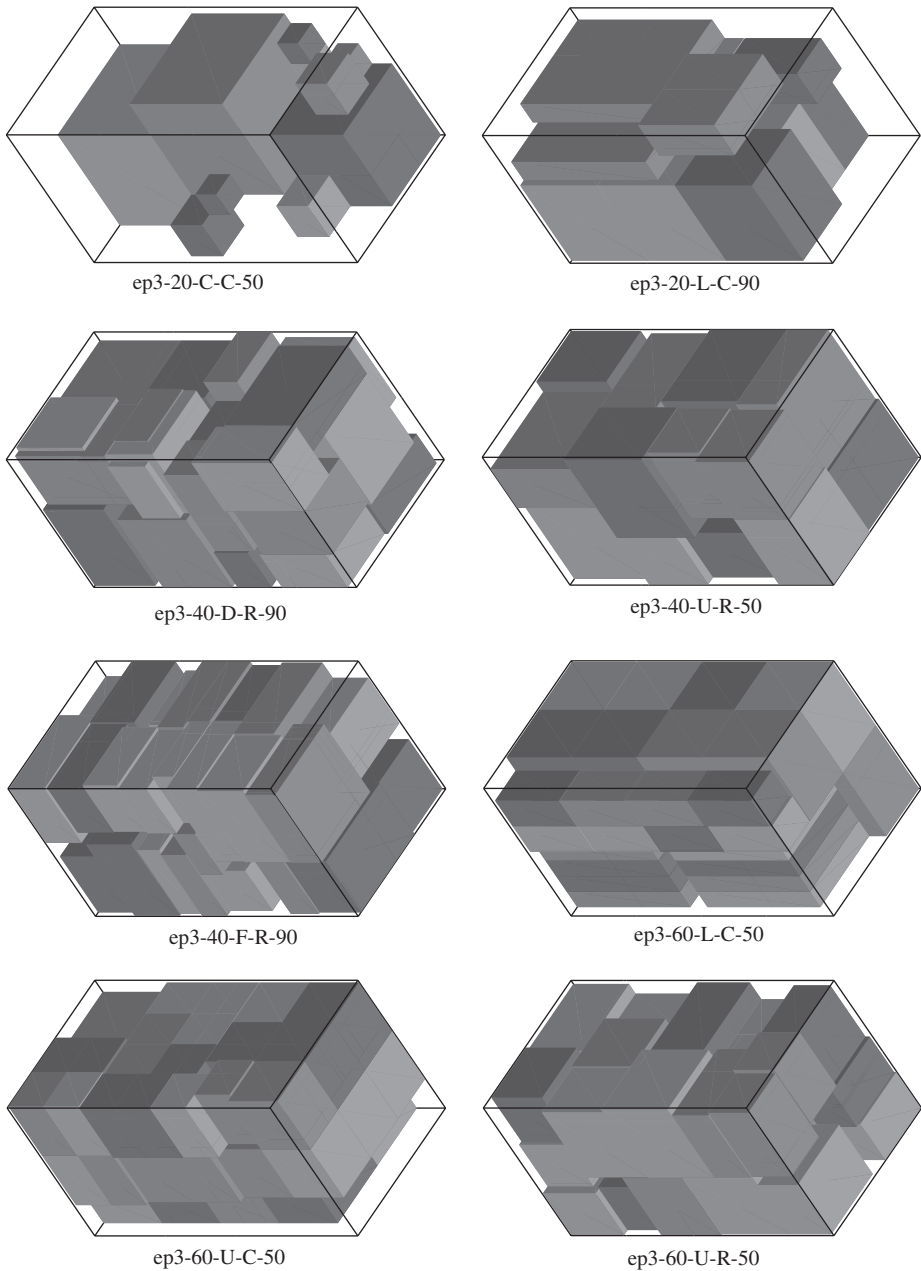


Fig. 10. Best results for eight three-dimensional instances.

bound thus finding optimal solutions. For the instances with 40 items, 16 of the best solutions are within 15% of the upper bound and as much as 7 are within 10%. The average gap between best solutions and upper bound is 12.53%. For the instances with 60 items, the heuristic reaches best solutions which have a gap which is less than 20% to the

upper bound for all but one of the instances. For eight of the instances, the gap is less than 10% and the average gap is as low as 11.4%. The best results for eight of the instances are shown in Fig. 10.

Our method finds high quality solutions quickly with an average gap to the upper bound of only 12.8%. In most of the instances, the best solution is found long before the heuristic's time limit, so solutions may be significantly closer to the optimal value than the bounds indicate. Results with large gaps could be due to the geometry of the boxes which can make it hard to utilize the three-dimensional knapsack as well as in the two-dimensional problems. Another explanation could be that the conservative scales bound does not function well on those instances or that the heuristic simply performs poorly in some cases.

To the best of our knowledge no other authors report gaps to upper bound for the three-dimensional knapsack packing problem. Fekete et al. [9] report only the number of problems solved to optimality. Their items are also larger than ours and we suspect that far fewer may be loaded in the knapsack than in our instances. This reduces the solution space and increases the strength of the bounds, making the instances easier to solve than the instances considered herein.

Instances for the Container Loading Problem contain hundreds of items, and state-of-the-art heuristics (e.g. [34–36]) reach volume utilization of slightly more than 90% on average. Initial experiments with our heuristic for container loading instances resulted in solutions with utilization of around 84% within 1 min.

Container loading heuristics however only optimize volume utilization, and hence cannot handle a general profit objective. Moreover, they exploit that most container loading instances contain many similar items. The strength of our heuristic is that it is not based on assumptions on the item sizes or profit. Based on our experiments we believe our heuristic is most suitable for medium sized instances where less than 80 items can fit in the knapsack simultaneously. We also suspect that the proposed heuristic for 3DKP cannot compete with heuristics tailored specifically for the Container Loading Problem.

7. Conclusion

In this paper, we have presented simulated annealing based approaches for the two- and three-dimensional knapsack problem. For the two-dimensional knapsack problem, we utilize an abstract representation for rectangle packings called sequence pair whereas for the three-dimensional problem we utilize a novel abstract representation for box packings called sequence triple. We have proved that the sequence triple is able to represent any fully robot packable packing.

The heuristic for two dimensions is generally able to reproduce the results of exact algorithms with similar running times. The heuristic also gives the best known results for the only unsolved classical-instance; `gcut13`. To demonstrate the high quality of the results of the heuristic for larger instances we have created a new set of instances with up to 200 rectangles and also here the heuristic performs extremely well by generating results with an average gap to our upper bound of less than 3.0%.

The heuristic for the three-dimensional case demonstrates the potential for the sequence triple representation. Exact methods are capable of solving small problems to optimality and tailored heuristics based on greedy principles work well for container loading problems. The proposed local search based heuristic performs very well for medium sized problems with an average gap to the upper bound of only 13%.

The heuristics are generally able to return very good results for both two- and three-dimensional problems within few minutes, and often within seconds for the classical two-dimensional benchmark instances.

Acknowledgments

The authors wish to thank the anonymous reviewers for their many helpful and supporting comments.

References

- [1] Fekete SP, Schepers J. A new exact algorithm for general orthogonal d -dimensional knapsack problems. In: Algorithms ESA '97, Springer lecture notes in computer science, vol. 1284, 1997. p. 144–56.
- [2] Pisinger D, Sigurd M. The two-dimensional bin packing problem with variable bin sizes and costs. *Discrete Optimization* 2005;2:154–67.
- [3] Pisinger D, Sigurd M. Using decomposition techniques and constraint programming for solving the two-dimensional bin packing problem. *INFORMS Journal on Computing* 2007;19:36–51.
- [4] Wascher G, Haussner H, Schumann H. An improved typology of cutting and packing problems. *European Journal of Operational Research*, 2007. Available online.

- [5] Beasley JE. Algorithms for two-dimensional unconstrained guillotine cutting. *Journal of the Operational Research Society* 1985;36:297–306.
- [6] Hadjiconstantinou E, Christofides N. An exact algorithm for general, orthogonal, two-dimensional knapsack problems. *European Journal of Operational Research* 1995;83:39–56.
- [7] Boschetti MA, Hadjiconstantinou E, Mingozzi A. New upper bounds for the two-dimensional orthogonal cutting stock problem. *IMA Journal of Management Mathematics* 2002;13:95–119.
- [8] Fekete SP, Schepers J. On more-dimensional packing III: exact algorithms. *Discrete Applied Mathematics*, 1997, submitted for publication.
- [9] Fekete SP, Schepers J, van der Veen JC. An exact algorithm for higher-dimensional orthogonal packing. *Operations Research* 2007;3(55): 569–87.
- [10] Baldacci R, Boschetti MA. A cutting-plane approach for the two-dimensional orthogonal non-guillotine cutting problem. *European Journal of Operational Research* 2006. Available online.
- [11] Caprara A, Monaci M. On the 2-dimensional knapsack problem. *Operations Research Letters* 2004;1(32):5–14.
- [12] Martello S, Monaci M, Vigo D. An exact approach to the strip packing problem. *INFORMS Journal on Computing* 2003;3(15):310–9.
- [13] Lai KK, Chan JWM. Developing a simulated annealing algorithm for the cutting stock problem. *Computers and Industrial Engineering* 1997;32:115–27.
- [14] Lai KK, Chan JWM. A evolutionary algorithm for the rectangular cutting stock problem. *International Journal on Industrial Engineering* 1997;4:130–9.
- [15] Leung TW, Yung CH, Troutt MD. Applications of genetic search and simulated annealing to the two-dimensional non-guillotine cutting stock problem. *Computers and Industrial Engineering* 2001;40:201–4.
- [16] Leung TW, Yung CH, Troutt MD. Application of mixed simulated annealing-genetic algorithm heuristic for the two-dimensional orthogonal packing problem. *European Journal of Operational Research* 2003;145:530–42.
- [17] Jakobs S. On genetic algorithms for the packing of polygons. *European Journal of Operational Research* 1996;88:165–81.
- [18] Beasley JE. A population heuristic for constrained two-dimensional non-guillotine cutting. *European Journal of Operational Research* 2004;156:601–7.
- [19] Alvarez-Valdes R, Parreño F, Tamarit JM. A grasp algorithm for constrained two-dimensional non-guillotine cutting problems. *Journal of Operational Research Society* 2005;56:414–25.
- [20] Alvarez-Valdes R, Parreño F, Tamarit JM. A tabu search algorithm for two-dimensional non-guillotine cutting problems. Technical Report TR07-2004, Universitat de Valencia; 2004.
- [21] Murata H, Fujiyoshi K, Nakatake S, Kajitani Y. VLSI module packing based on rectangle-packing by the sequence pair. *IEEE Transaction on Computer Aided Design of Integrated Circuits and Systems* 1996;15:1518–24.
- [22] Tang X, Tian R, Wong DF. Fast evaluation of sequence pair in block placement by longest common subsequence computation. In: *Proceedings of DATE 2000 (ACM)*. France: Paris; 2000. p. 106–10.
- [23] Tang X, Wong DF. Fast-sp: a fast algorithm for block packing based on sequence pair. In: *Asia and South Pacific design automation conference*, 2001.
- [24] Pisinger D. Denser packings obtained in $O(n \log \log n)$ time. *INFORMS Journal on Computing* 2007;19:395–405.
- [25] Kellerer H, Pferschy U, Pisinger D. *Knapsack problems*. Berlin, Germany: Springer; 2004.
- [26] Dembo RS, Hammer PL. A reduction algorithm for knapsack problems. *Methods of Operations Research* 1980;36:49–60.
- [27] Martello S, Pisinger D, Vigo D, den Boef E, Korst J. Algorithm 864: general and robot-packable variants of the three-dimensional bin packing problem. *ACM Transactions on Mathematical Software* 2007;33:1–7.
- [28] Fekete SP, Schepers J. A general framework for bounds for higher-dimensional orthogonal packing problems. *Mathematical Methods of Operations Research* 2004;60:311–29.
- [29] Beasley JE. An exact two-dimensional non-guillotine cutting tree search procedure. *Operations Research* 1985;33:49–64.
- [30] Christofides N, Whitlock C. An algorithm for two dimensional cutting stock problems. *Operations Research* 1977;25:30–44.
- [31] Wang PY. Two algorithms for constrained two dimensional cutting stock problems. *Operations Research* 1983;31:573–86.
- [32] Hifi M. Two-dimesional (un)constrained cutting stock problems, (<http://www.laria.u-picardie.fr/hifi/OR-Benchmark/2DCutting/>); 2006.
- [33] Pisinger D. A minimal algorithm for the 0–1 knapsack problem. *Operations Research* 1997;45:758–67.
- [34] Moura A, Oliveira JF. A grasp approach to the container-loading problem. *IEEE Intelligent Systems* 2005;20(4):50–7.
- [35] Mack D, Bortfeldt A, Gehring H. A parallel hybrid local search algorithm for the container loading problem. *International Transaction in Operations Research* 2004;11:511–33.
- [36] Pisinger D. Heuristics for the container loading problem. *European Journal of Operations Research* 2002;3(141):382–92.